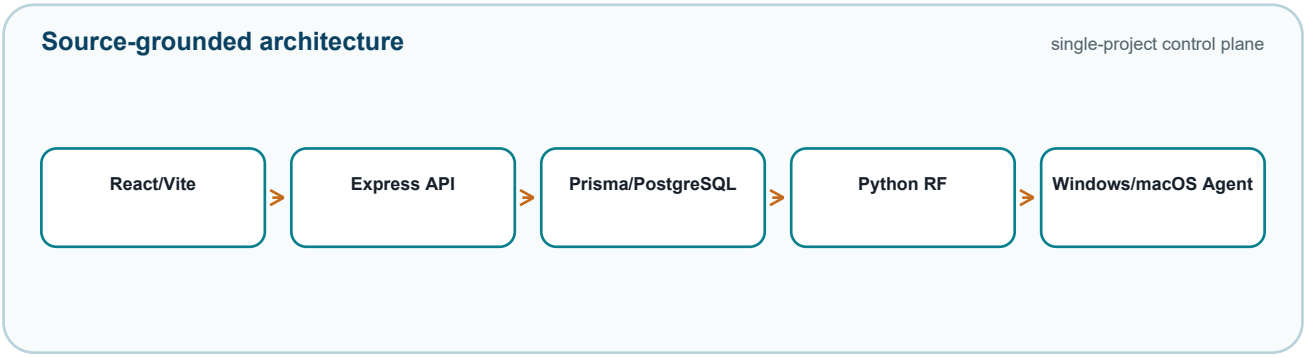
CYBERORBIT

Complete Technical Documentation

AI-Powered Zero Trust and User and Entity Behavior Analytics (UEBA) Platform



Prepared for academic final review, technical demonstration and viva preparation

Repository evidence audit: 4 November 2026

This PDF describes the checked-in implementation and explicitly separates implemented, automated-test verified, research, simulated, unsupported and unverified behavior. It does not certify production security or physical endpoint enforcement.

Table of contents

The contents are generated from the numbered chapters and pipeline sections below. A detailed 100-question viva section is appended from the verified generated question bank.

Table of contents 2
1. Project introduction and problem statement 4
2. Complete system architecture 5
3. Technology stack 6
4. Frontend architecture 7
5. Backend, APIs and request lifecycle 8
6. Database architecture 9
7. Machine learning fundamentals and actual implementation 10
8. CERT model, dataset evidence and 21-feature contract 11
9. Cybersecurity, Zero Trust, UEBA and risk 12
10. Windows and macOS endpoint agents 13
11. Account containment and restoration 14
12. Alerts, incidents, simulator and real-time updates 15
13. Complete end-to-end pipelines 16
14. Testing, deployment, limitations and future scope 20
15. Glossary and two-minute revision 21
Appendix: evidence paths 22

1. Project introduction and problem statement

CYBERORBIT is a security operations and policy-enforcement prototype for administrator-led monitoring of identities, devices and protected-resource access.

1.1 Problem and motivation

Traditional perimeter security assumes that a user inside a network or holding a valid credential is trustworthy. That assumption is weak against stolen credentials, compromised endpoints, insider misuse, unusual access and policy violations. CYBERORBIT moves the decision point toward continuous verification: the server checks the authenticated administrator/session, company scope, resource sensitivity, user role, device posture, current observations, historical baseline, trust and incident holds.

1.2 UEBA and Zero Trust in this project

UEBA means User and Entity Behavior Analytics. User behavior is employee activity; entity behavior includes devices and endpoint state. Zero Trust means never accepting implicit trust: verify identity, device and authorization, use least privilege, evaluate current context and require step-up controls for sensitive actions. The repository implements these ideas through MFA-verified admin sessions, scoped database queries, enrolled endpoints, risk-adaptive action bands, trust history, approval workflows and signed endpoint commands.

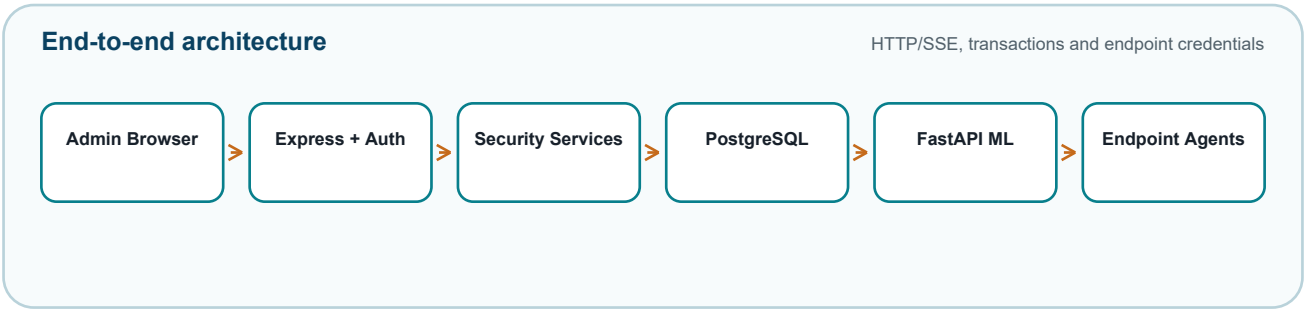
1.3 Roles and scope

The Prisma enum defines EMPLOYEE, MANAGER, SECURITY_ADMIN and SYSTEM_ADMIN. The administrator console is the active user-facing surface. The current API rejects employee console login, so employee access decisions are exercised through server-side evaluation and tests rather than a production employee session. The scope includes users, resources, endpoints, telemetry, incidents, notifications, risk/trust, policies, simulation and containment. Enterprise SSO, hardware MFA, external resource gateways, distributed workers and immutable audit archival are not implemented integrations.

Primary evidence: README.md; server/routes/index.js; server/middleware/auth.js; prisma/schema.prisma; server/services/security-service.js.

2. Complete system architecture

The system is a coordinated browser, API, database, ML and endpoint-agent architecture.



2.1 Component responsibilities

The React/Vite client renders dashboards and actions. Express owns HTTP lifecycle, middleware and route dispatch. Security services own authorization, risk, policy, incident, notification and containment logic. Prisma provides relational access to PostgreSQL. EndpointService owns assignment, enrollment, heartbeat, ingestion, endpoint dashboard and command state. Python FastAPI loads the preserved Random Forest and emits inference/XAI output only; it cannot authorize by itself. The agent collects observations, encrypts queued events, retries uploads and acknowledges commands.

2.2 Communication and data flow

Browser requests carry a short-lived MFA-verified admin JWT; refresh state uses an HTTP-only cookie. Agents use separate enrollment/device credentials. Endpoint events carry version, event ID, device ID, occurrence time and metadata. Accepted rows are persisted in EndpointEvent, while security assessments create ActivityEvent, RiskScore, TrustHistory, Incident, AccessRequest, SecurityNotification and AuditLog records as applicable. SSE broadcasts scoped changes; polling recovers persisted notifications.

Decision/data	Responsible component	Evidence
Browser authentication	auth.js + session-service.js	JWT, MFA, persisted Session
Endpoint identity	endpoint-service.js	EndpointDevice credential/assignment
Risk score	cyber/risk-engine + security-service	Context factors and optional RF evidence
Policy action	policy-service.js	Company ZeroTrustPolicy
Durable state	Prisma/PostgreSQL	schema.prisma and migrations
Model inference	ml/src + ml-service.js	Exact 21-feature contract

2.3 Deployment

Production shape is a trusted HTTPS proxy in front of Node, PostgreSQL as durable storage, built client assets served by Node, and a private/internal Python inference process. Local setup uses npm, Prisma migrations, Vite and Uvicorn. Endpoint production requires trusted HTTPS; loopback HTTP is an explicit development exception.

Evidence: server/server.js; server/routes/index.js; server/services/endpoint-service.js; server/services/sse.js; server/models/prisma.js.

3. Technology stack

Versions below are declared direct dependencies or runtime requirements; indirect package versions are not presented as direct usage.

Layer	Technology/version	Actual usage
Frontend	React ^19.0.1, React DOM ^19.0.1	JSX pages, components, hooks and context
Build	Vite ^6.2.3, @vitejs/plugin-react ^5.0.4	Development middleware and production bundle
UI	Tailwind ^4.1.14, lucide-react, react-icons, motion	Styling, icons and transitions
Charts	Recharts ^3.10.1	Dashboard/risk/analytics visualizations
Backend	Node ES modules, Express ^4.21.2	API, middleware and SPA hosting
Database	PostgreSQL	Durable relational system of record
ORM	Prisma ^6.4.1, @prisma/client ^6.4.1	Schema, migrations, queries and transactions
Auth	Node crypto, JWT HS256, bcrypt/bcryptjs	Tokens, hashes, MFA/session security
ML	FastAPI 0.141.1, Uvicorn 0.53.0	Python health and prediction API
ML data	NumPy 2.1.3, pandas 2.2.3	Numerical validation and dataframes
ML model	scikit-learn 1.6.1, joblib 1.4.2	Random Forest loading/inference
XAI	SHAP 0.52.0	TreeExplainer reasons for complete aggregates
Agent/native	Node, PowerShell, C#, Swift, Python, C++/CMake	Collectors, service, packaging and OS adapters
Testing	node:test, Python unittest, browser harness	Backend, agent, ML and UI contract validation

Evidence: package.json; client/package.json; server/package.json; agent/package.json; ml/requirements.txt; source imports under client/src, server, agent and ml/src.

4. Frontend architecture

The client is a React/Vite administrator console composed through App.jsx rather than a declared routing library.

4.1 Shell, state and API integration

client/src/App.jsx loads `/api/auth/me`, initializes scoped users/incidents/resources/devices/activity/stats, owns active page state, sidebar/theme/session state, notifications and SSE. client/src/services/api.js and session.js attach tokens and perform refresh handling. The backend remains the authorization boundary; client state only controls presentation and request initiation.

Page	Purpose and actual backend surface
LoginPage	Admin password, OTP, resend and reset: /api/auth/*
DashboardPage	Summary, risk distribution/trends and activity: /api/dashboard/*
EmployeeManagementPage / UsersPage	Users, assignments, freeze/unfreeze/baseline: /api/users/*
EmployeeDashboard	Scoped employee details and risk analysis: /api/users/:id, /api/risk/analysis/:id
DevicesPage / EndpointMonitor	Logical device and endpoint dashboard/actions: /api/devices, /api/endpoint/*
IncidentsPage / AlertsPage	Incident list/export/explain/resolve and containment actions
PoliciesPage	Policy read, preview, save and history
AiRiskAnalysisPage / ExplainableAiPage	Range-based evidence and explanations
MlAnalyticsPage	Model metadata, prediction history and guarded prediction
SimulatorPage	Synthetic scenarios/custom route and history
AccessRequestsPage / ResourcesPage	Requests, approvals and protected resources
AuditLogsPage / AdminManagementPage / SettingsPage	Audits, admin/local-operation review and sessions/password

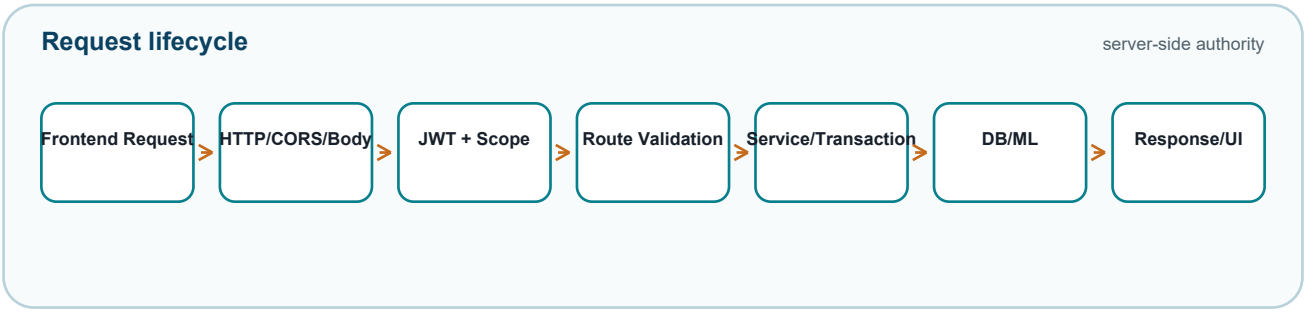
4.2 Notifications, loading and errors

The admin bell fetches persisted notifications at login, visibility changes and a 60-second interval, merges SSE records by notification ID, and calls recipient-scoped read/read-all routes. Pages use loading/error/empty states and do not fabricate risk or alerts when refresh fails. Charts transform backend arrays; they do not become a second risk engine.

Evidence: client/src/App.jsx; client/src/pages; client/src/components/TopBar.jsx, Sidebar.jsx, EndpointMonitor.jsx; client/src/services/api.js and session.js.

5. Backend, APIs and request lifecycle

The backend is an Express ES-module application with mounted route groups and domain services.



5.1 Startup and middleware

server/server.js loads environment variables, applies readiness gating, security HTTP policy, CORS, bounded parsers, endpoint request context, `/health`, `/api`, Vite development middleware or production static hosting, retention scheduling, daily snapshot refresh and error handlers. routes/index.js mounts endpoint/auth publicly, then requires authentication for the console and adds security-admin/main-admin gates on sensitive groups.

API group	Representative active routes	Authorization
Auth	/auth/login, /auth/mfa/verify, /auth/refresh, /auth/logout, /auth/sessions	Public/challenge or authenticated
Users/admins	/users, /admins, assignment-options	Security admin/main admin/scoped
Endpoint	/endpoint/assign, enroll, heartbeat, ingest, dashboard, config, revoke	Endpoint credential or admin
Risk/policy	/risk/*, /trust/*, /policies, /policies/preview	Scoped/security admin
Resources/access	/resources, /resources/:id/access, /access-requests/*	Authenticated/reviewer/admin
Incidents/audit	/incidents, export, resolve, /audit-logs	Investigator/security admin
ML/simulation	/ml/*, /simulation/*, /copilot/chat	Security admin
Realtime/system	/events/stream-session, /events/stream, /system/*	Authenticated/admin stream

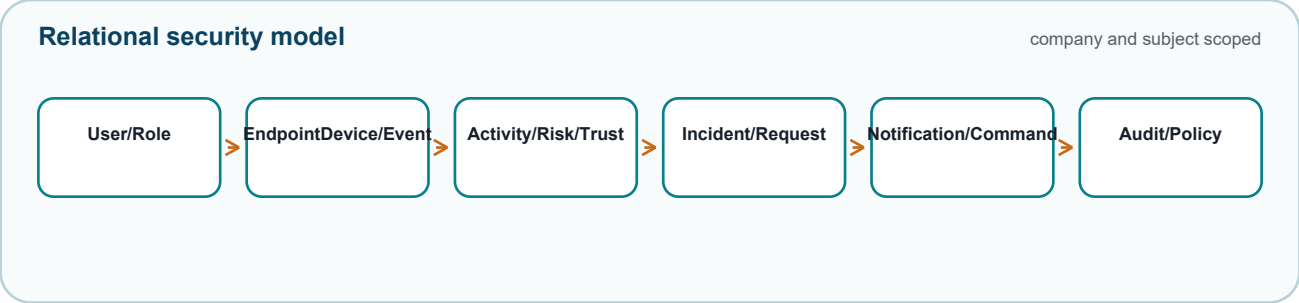
5.2 Validation and errors

auth.js validates JWT signature, algorithm, expiry, user status, MFA, password/auth version and persisted session. security-common.js provides SecurityError, API wrappers, company/subject scope and audit helpers. EndpointService validates credentials, assignment, timestamps, event IDs, versions and payload limits. ML errors include incomplete data, schema mismatch, service unavailable and model-version mismatch. Errors are structured and redacted.

Evidence: server/server.js; server/routes/index.js; server/middleware/auth.js; server/middleware/security-http.js; server/services/security-common.js; server/services/endpoint-service.js; server/services/ml-service.js.

6. Database architecture

Prisma schema and migrations define the durable security and endpoint data model.



Model	Purpose and important relationships
User	Identity, company, role, manager/admin assignment, status, risk/trust and auth version
EndpointDevice	Enrolled endpoint, platform, mapped SID/UID, credential/enrollment hashes, health/config and lifecycle
EndpointEvent	Versioned device event; unique deviceId+eventId; userId may be null for device-level evidence
Device	Logical/browser posture, trust score/status/fingerprint and activity relation
ActivityEvent	User/device/resource activity, type, severity, evidence flags and optional incident relation
BehaviorProfile	Prior 30-day baseline hours, volume, locations and sample statistics
RiskScore/TrustHistory/DailyRiskSnapshot	Current and historical risk/trust plus UTC daily rollups
Incident/AccessRequest	Investigation and protected resource approval workflows
SecurityNotification	Recipient/company scoped in-app alert with dedupeKey and readAt
EndpointEnforcementCommand	Signed BLOCK/ALLOW or agent operation state, expiry and acknowledgment
Session/MfaSession/PasswordResetToken	Auth/session lifecycle
AuditLog/ZeroTrustPolicy	Application audit trail and company policy
ModelVersion/MlPrediction	Model metadata, feature vector, score, class and attributions

6.1 Integrity and isolation

Unique constraints protect user identity, endpoint credentials, activity IDs, endpoint event IDs, commands, incidents, requests and recipient/dedupe notifications. Indexes support company, subject, timestamp, status, risk/trust and unread queries. Company and assignment filters are applied in services and routes; this is not merely a UI filter. PostgreSQL administrators can still modify storage, so immutable archival is future scope.

Evidence: prisma/schema.prisma; migrations under prisma/migrations; server/models/prisma.js; endpoint-retention.js; daily-risk-snapshot-service.js.

7. Machine learning fundamentals and actual implementation

The PDF distinguishes ML theory from the exact repository behavior.

7.1 Fundamentals

AI is the broad field of machine-supported reasoning; ML learns patterns from data. Supervised learning uses labels; unsupervised learning seeks structure without labels. Classification selects a class; regression predicts a continuous value; anomaly detection highlights unusual observations. Features are inputs, labels are outcomes, training fits a model, validation supports choices, testing evaluates held-out data, and inference uses a saved model. Overfitting learns noise; underfitting misses structure; leakage uses information unavailable at decision time. Precision measures how many positive predictions are correct; recall measures how many real positives were found; F1 balances both; a confusion matrix contains TP, FP, TN and FN. In security, false positives waste investigator time and false negatives miss threats.

7.2 Random Forest

A Random Forest combines many decision trees. Each tree sees a bootstrap sample and random candidate features at splits; the ensemble aggregates class votes/probabilities. The repository metadata/research audit records 150 trees, max depth 14, minimum split 6, minimum leaf 2, balanced classes and seed 42. Python loads the joblib artifact, forces n_jobs=1 and calls predict_proba for class 1. The result is an uncalibrated positive-class score, not a posterior probability.

7.3 Isolation Forest status

Isolation Forest theoretically isolates unusual points by random partitions; short path lengths indicate easier isolation. The active Python service does not load or call an Isolation Forest. Legacy names such as anomalyScore or unsupervisedAnomalyScore are compatibility/historical fields and must not be described as live Isolation Forest evidence.

7.4 Artifact and inference

ml/src/inference.py loads model, schema and metadata; validates exact names, nonnegative finite values, calendar consistency, integer counts, totals, cross-source activity and bounded distinct counts; then returns prediction, threshold, model/schema fingerprints, aggregation level and provenance. ml/src/app.py exposes `/health` and `/predict`. server/services/ml-service.js validates Node-side contract and rejects mismatches. ml/src/xai.py uses SHAP TreeExplainer for top five feature reasons.

Saved item	Repository evidence
Model	ml/models/multi_source_ueba_rf.joblib
Schema	21 ordered names in multi_source_feature_schema.json
Metadata	CERT r4.2, user-day, threshold 0.85 and saved metrics
Inference	FastAPI /predict plus Node fingerprint/provenance checks
XAI	SHAP TreeExplainer for complete aggregate only
Live endpoint RF	Not implemented/promoted; readiness is insufficient

8. CERT model, dataset evidence and 21-feature contract

The preserved model is a CERT-style multi-source user-day model, not a direct endpoint model.

8.1 Dataset evidence and metrics

The model metadata names CERT Insider Threat Dataset r4.2 with logon, device, file, http and email sources. The notebook records chronological 70/15/15 row splitting. Raw source files, release README and authorized answer-key archive are absent, so saved metrics are not independently reproducible from this checkout. Metadata reports train/validation/test rows 231,316/49,568/49,568; test threats 149; accuracy 0.997014; precision 0.507042; recall 0.241611; F1 0.327273; ROC AUC 0.967034; average precision labelled PR_AUC 0.350811; FPR 0.000708; FNR 0.758389; TP 36, FP 35, TN 49,384, FN 113.

Feature	Meaning and calculation
logon_count	Logon rows per user-day
device_event_count	Device rows per user-day
file_event_count	File rows per user-day
http_event_count	HTTP rows per user-day
email_event_count	Email rows per user-day
day_of_week / is_weekend	Monday-zero calendar fields; weekend derives from weekday >=5
logon/device/file/http/email_unique_pc_count	Distinct source PC counts per day
unique_files	Distinct file.filename values
file_content_events	Non-null file content row count
unique_urls	Distinct HTTP URL values
unique_http_content	Distinct non-null HTTP content
unique_recipients	Distinct email.to values
email_size_total	Sum of email.size
attachments_total	Sum of email.attachments
total_activity	Sum of five source event counts
cross_source_activity	Number of source families with count > 0

8.2 Preprocessing and limitations

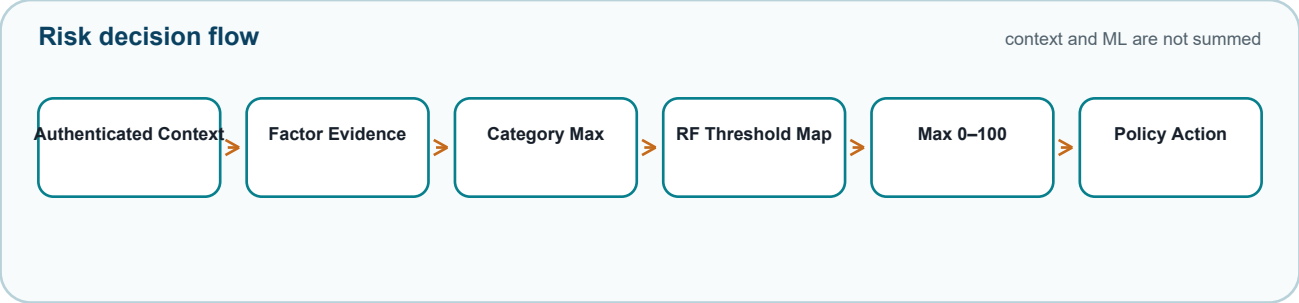
Notebook aggregation outer-joins source families and fills absent family joins with zero for a fully observed training row. The saved inference path has no fitted scaler, encoder or imputer. That does not justify zero-filling endpoint gaps: current readiness rejects incomplete/semantically incompatible data. The staging/research pipeline performs timestamp, duplicate, ordering, target-column, availability and split checks, but cannot validate absent official source semantics.

8.3 Endpoint compatibility

The repository audit rates verified live endpoint compatibility as 0/21. Endpoint process, USB, file-change, TCP, session and application observations are adjacent evidence but not equivalent CERT source rows. Email, HTTP history/content, file content and CERT PC semantics are unavailable. No endpoint event is promoted into the CERT vector; test-ml-feature-aggregator.js intentionally rejects incomplete pseudo-aggregation.

9. Cybersecurity, Zero Trust, UEBA and risk

CYBERORBIT combines identity security, device security, behavior context and policy enforcement.



9.1 Security controls

Authentication uses password hashes, OTP/MFA, short-lived HS256 JWTs, HTTP-only refresh cookies, refresh rotation, persisted sessions and auth-version invalidation. Authorization uses roles, company filters, assigned administrator/manager relationships, resource required roles and sensitivity. Endpoint security uses hashed credentials, enrollment expiry, revocation, event IDs, signed expiring commands and replay journals. Audit rows preserve application actions; PostgreSQL administrators remain a storage trust boundary.

9.2 UEBA baseline and observations

ueba-service.js learns prior 30-day daily download totals, login-hour quantiles and typical locations, excludes the current day and requires at least seven observed days. The engine can use volume, learned hours/locations, auth failures, privilege change, explicit USB transfer, untrusted device posture, cross-department access and unresolved incidents. Ordinary process activity, attached USB, file changes or TCP connections do not automatically become threats.

9.3 Exact live risk formula

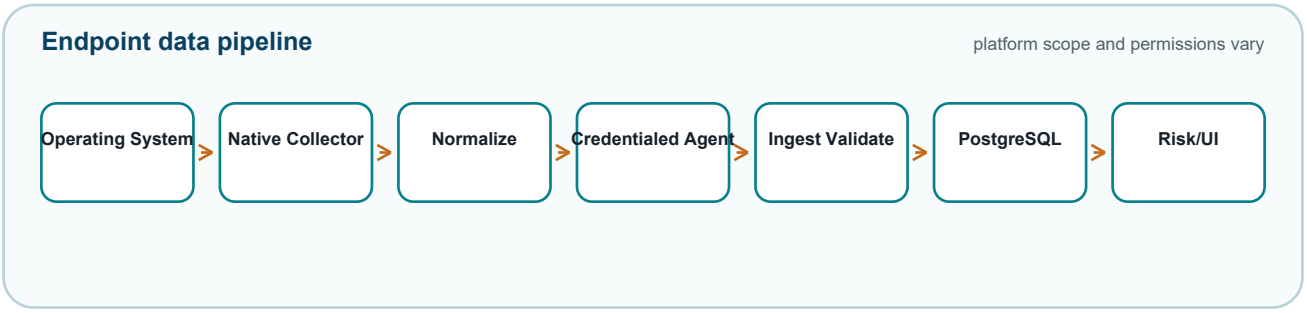
The authoritative formula is contextualEvidence = min(100, sum of the strongest factor in each category). If a valid RF score p and operating threshold t exist, then p <= t maps to M*p/t and p > t maps to M + (100-M)*(p-t)/(1-t), where M is the MFA boundary. Final riskScore = round(clamp(max(contextualEvidence, mlEvidence or 0),0,100)). The old 70/30 calculation in ml/src/risk.py is not the backend authorization formula.

9.4 Actions and explainability

Documented defaults are 0–39 ALLOW/log, 40–59 MONITOR, 60–74 MFA, 75–89 reviewed approval and 90–100 FREEZE if enabled; stored company policy can customize. Factor descriptions form rule-based explanations. Complete RF aggregate results can include SHAP reasons. Incident AI explanation returns stored policy-factor text and is advisory; it is not automatically SHAP.

10. Windows and macOS endpoint agents

The agent is separate from the website and packaged with a runtime for employee installation.



10.1 Windows

The Windows package uses a service host and Node agent. Enrollment maps a verified SID and exchanges a one-use token; the agent stores protected credentials and an encrypted durable queue. Collectors emit process/session/USB/network/file/application/health observations. Windows PnP/CIM classification can distinguish storage/peripheral categories where the OS reports them. Heartbeats deliver configuration and signed stop/start, USB-policy and account-control commands. The native account helper refuses unsafe administrator/built-in/domain/recovery mappings and reports BLOCKED, ALLOWED, LOGOFF_PENDING, EXTERNALLY_DISABLED, RECOVERY_REQUIRED, UNSAFE_* or error. Real two-account disable/reboot/recovery and physical USB acceptance are not verified here.

10.2 macOS

The employee path is a user-owned LaunchAgent with Keychain/configuration and UID-scoped collection. macos.js uses process/session/network observations and system_profiler SPUSBDataType inventory. Desktop and browser-title features depend on permissions/consent; Safari titles are unsupported. User mode cannot provide account-wide enforcement; an optional root/MDM path is separate and unvalidated. The current macOS-named artifact is a build kit, not an employee-installable release. Native launchd, Keychain, TCC, Swift helper and physical telemetry acceptance remain pending.

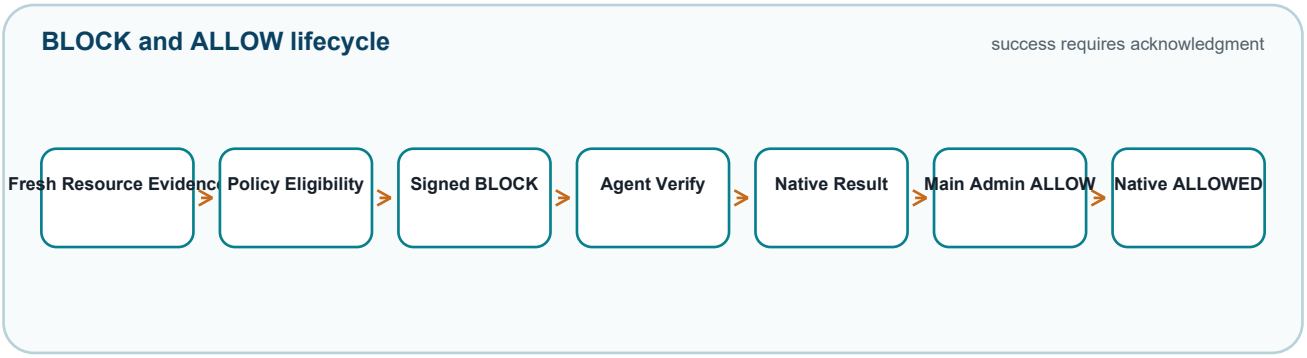
10.3 Queue and retry

EncryptedQueue persists event records; Uploader sends batches and acknowledges only accepted IDs, retrying after network failures. Heartbeat remains a control channel even when administrator monitoring pause suppresses collection/upload. Revoked credentials stop the service path. Offline cached telemetry does not authorize a new containment action.

Evidence: agent/src/service.js, enrollment.js, uploader.js, queue.js, collectors/*, account-control.js, remote-stop.js, native/*, packaging/*, agent/README.md.

11. Account containment and restoration

Application restriction and OS account enforcement are separate state machines.



11.1 Freeze eligibility

Automatic containment is explicit opt-in and requires fresh authenticated protected-resource access, contextual evidence at the freeze threshold, at least two independent non-history categories, current supported mapped Windows identity and required Main Admin policy state. Simulator, stale/backdated/imported, generic critical or unavailable-confidence evidence cannot authorize OS containment. Manual freeze still requires administrator workflow and audit.

11.2 BLOCK and restoration

The server persists risk/incident/audit and creates an expiring device/action/identity-bound command. A fresh heartbeat delivers it; the agent verifies signature, shape, expiry, mapping and replay state; native helper attempts the safe OS action; only an authenticated result can complete the backend state. Restoration requires incident review as configured, Main Admin fresh MFA, an ALLOW command and ALLOWED acknowledgment. Pending/offline/unsupported/failed/partial states never become a false success.

11.3 macOS boundary

A macOS employee LaunchAgent cannot enforce account-wide restriction in user mode. Any claim of universal macOS freeze would contradict the implementation; managed root/MDM deployment is future/independent work.

12. Alerts, incidents, simulator and real-time updates

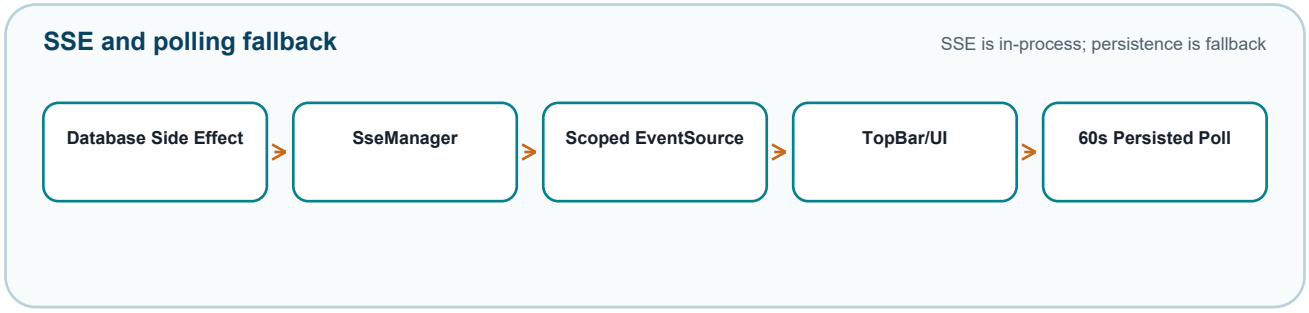
The system distinguishes observations, inferences, decisions and synthetic demonstrations.

12.1 Alert integrity

Endpoint identity and event validity are checked before notifications. SecurityNotification is company/recipient scoped and unique by recipient+dedupeKey. USB connection alerts are device-level, do not target an employee, omit raw hardware identifiers and do not claim file access or transfer. Incident notifications use incident dedupe keys. Authentication failures, every policy violation, trust change and all endpoint conditions are not automatically wired to the bell; coverage must be proven per path.

12.2 Simulator isolation

simulation.js validates bounded scenario/custom inputs and records SIMULATION_RUN audit history. Tests assert simulator calls do not create employee ActivityEvent, RiskScore, TrustHistory, Incident, endpoint command, real account status or OS action. Simulator output is synthetic and cannot be used as CERT performance or production telemetry evidence.



12.3 SSE

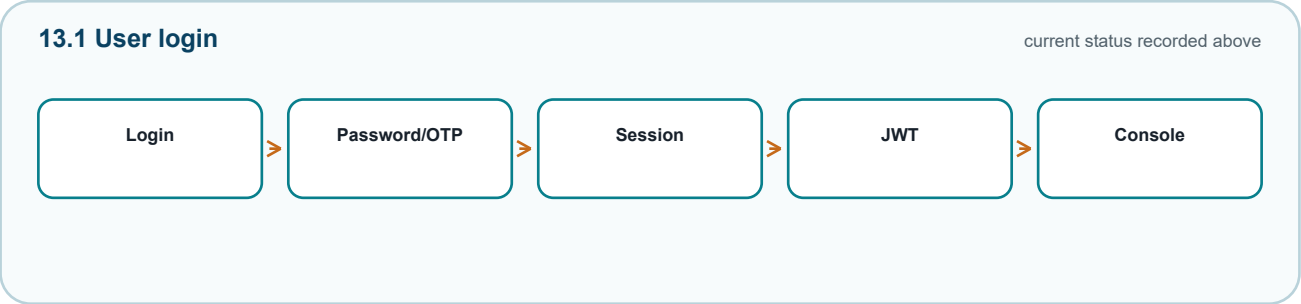
events.js and sse.js establish authenticated stream sessions, send CONNECTED and 25-second HEARTBEAT events, recheck active role/company/authVersion, and broadcast scoped activity, risk, trust, incident, device, policy, containment and notification events. The browser still polls persisted notifications at login, visibility and 60-second intervals. SSE is not a distributed pub/sub system.

13. Complete end-to-end pipelines

The following pipelines summarize trigger, input, validation, decision, persistence, output and status.

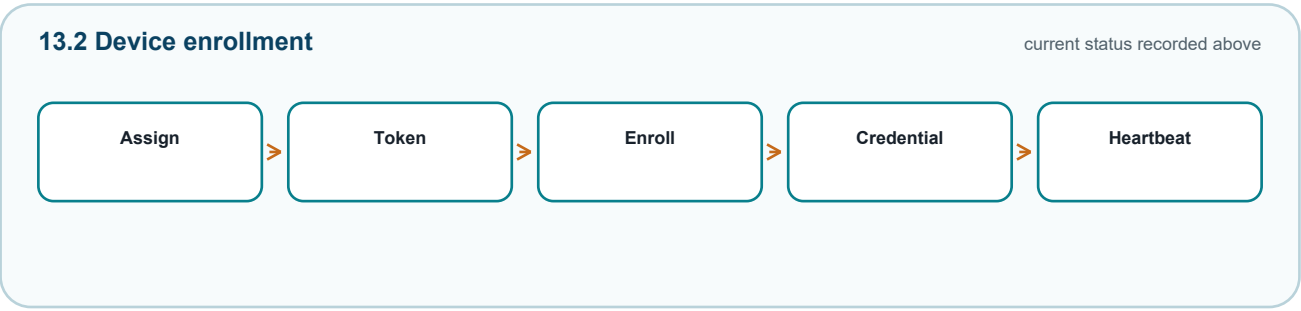
13.1 User login

Credentials + OTP → auth-service validates password/challenge → issueSession creates MFA-verified Session and JWT → browser stores access token/refresh cookie → /auth/me loads scope. Status: automated-test verified.



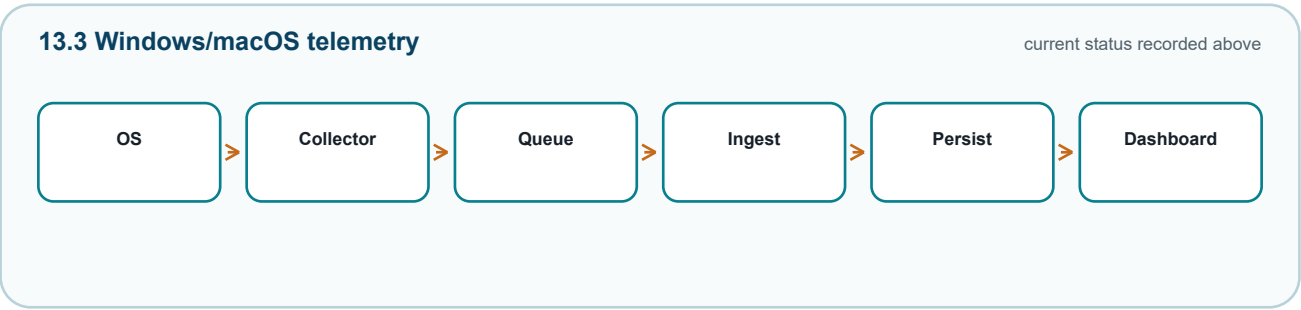
13.2 Device enrollment

Admin assignment → verified SID/UID + one-use token → agent hidden prompt → /endpoint/enroll exchanges token → hashed credential/device config persisted → heartbeat. Status: automated-test verified; physical install unverified.



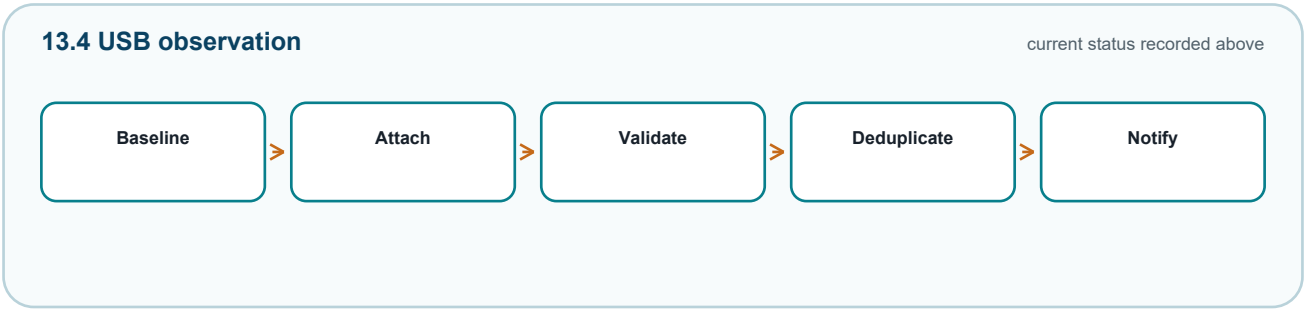
13.3 Windows/macOS telemetry

OS collector → normalizer → agent identity → encrypted queue → uploader → endpoint ingest validation → EndpointEvent → dashboard/risk coverage. Status: source/automated contract verified; platform physical coverage partial.



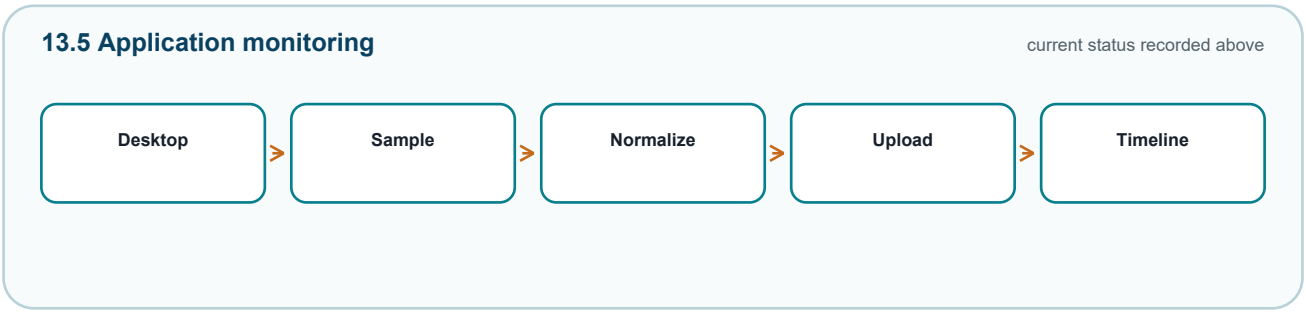
13.4 USB observation

Baseline inventory is suppressed → later attach creates USB_CONNECTED → authenticated event/ID/assignment validation → device-level row and scoped notification → removal permits later distinct attach. Status: automated-test verified; physical insertion unverified.



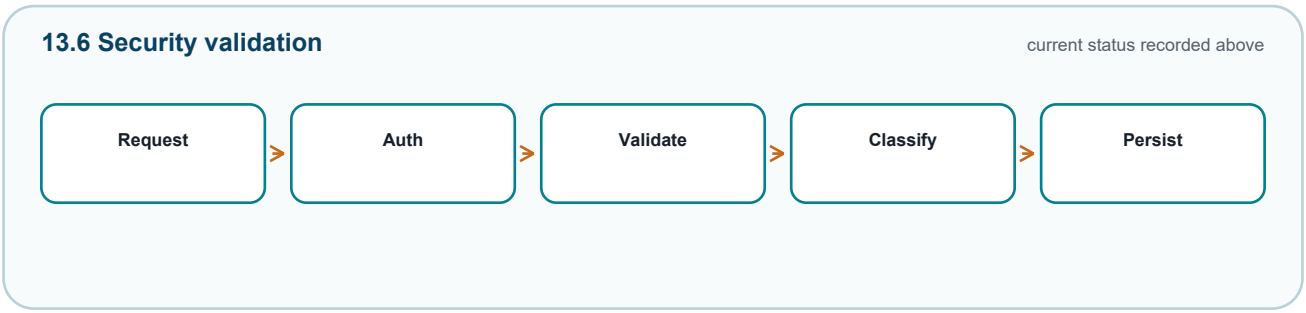
13.5 Application monitoring

Desktop sampling/helper → foreground identity and duration normalization → usage/process event → encrypted queue → backend timeline → UI card/chart. Status: automated-test verified; brief activity/OS identity can be missed.



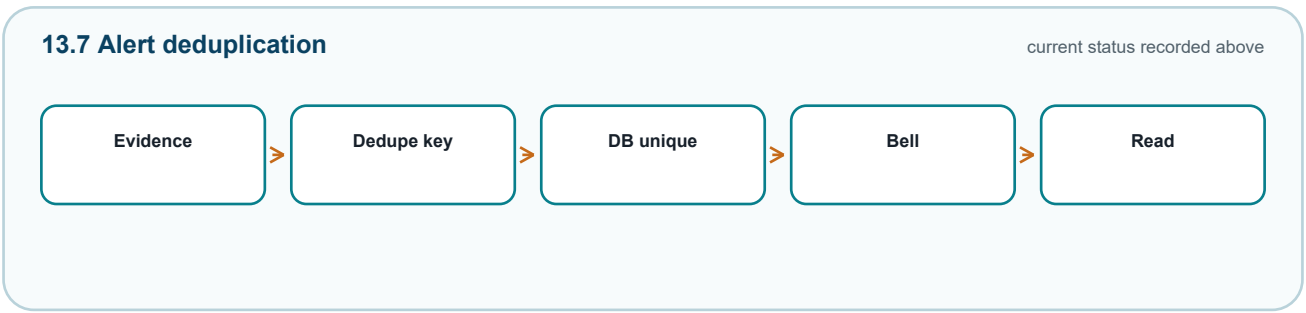
13.6 Security validation

Request/event → auth + company/resource/device checks → schema and evidence validation → classify factor/severity → persist or reject → response/audit. Status: automated-test verified.



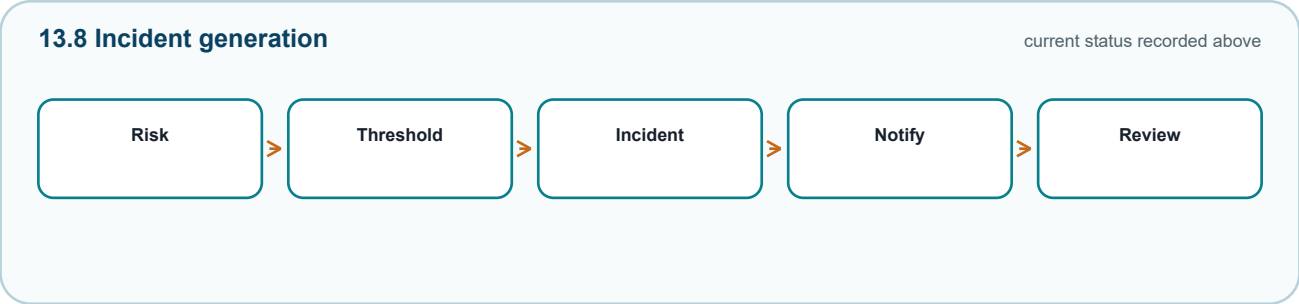
13.7 Alert deduplication

Accepted event/incident → dedupe key + unique constraint → create one recipient row → SSE/polling merge by ID → read state. Status: automated-test verified.



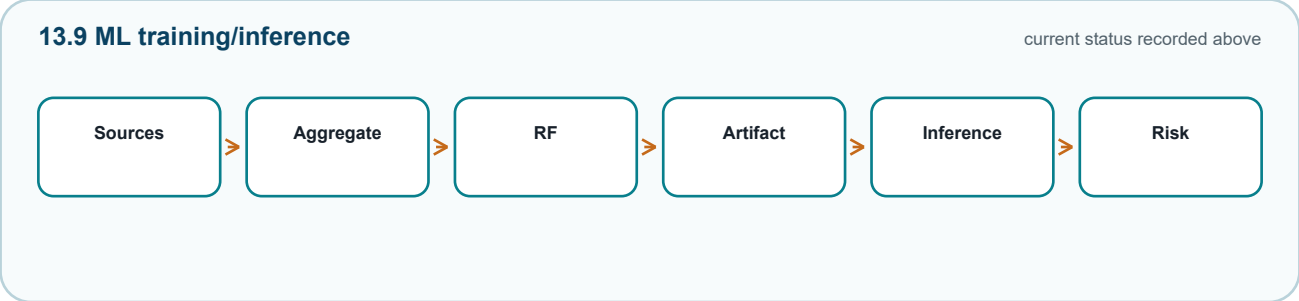
13.8 Incident generation

Qualifying risk/action → incident threshold and existing-ID check → Incident row with factors/explanation → security notifications/audit → admin review/resolve. Status: automated-test verified.



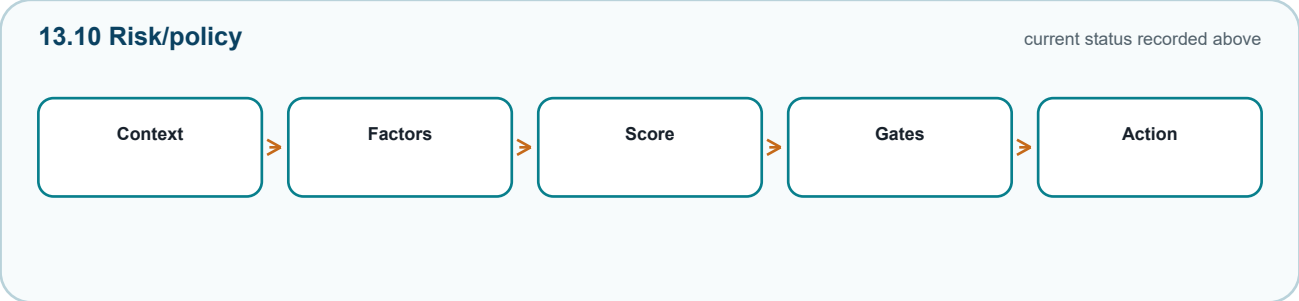
13.9 ML training/inference

CERT source families → user-day aggregation → contract validation → RF training/artifact → model load → complete aggregate /predict → threshold mapping → risk. Endpoint-native stages after endpoint collection are not implemented.



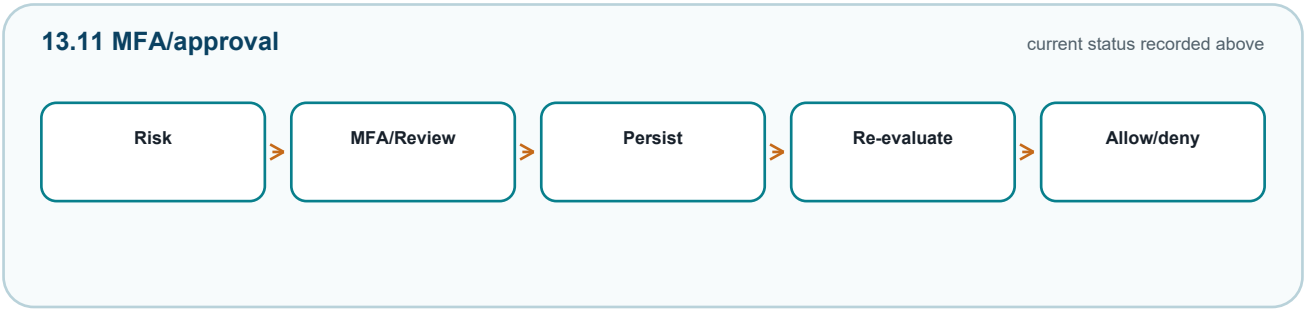
13.10 Risk/policy

Current authenticated context → contextual factors → strongest category factors → optional RF mapping → max score → trust/role/incident gates → action → history/audit. Status: automated-test verified.



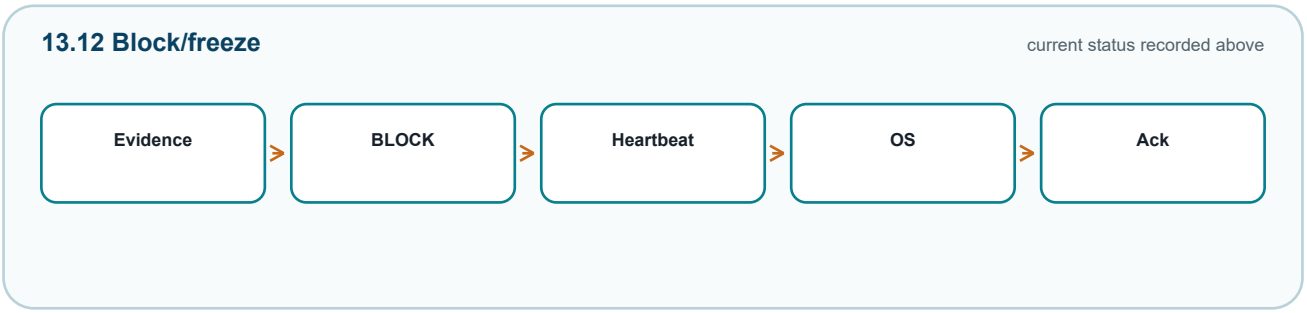
13.11 MFA/approval

MFA or approval action → fresh OTP or authorized reviewer → session/request status persisted → protected action re-evaluated → access/containment response. Status: automated-test verified.



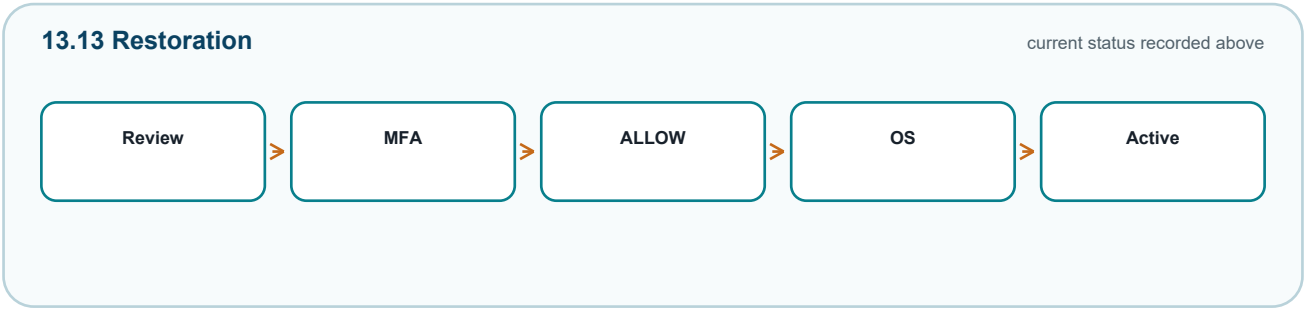
13.12 Block/freeze

Eligible fresh resource evidence → expiring signed BLOCK → heartbeat → agent/native helper → BLOCKED acknowledgment → account/session/incident state. Status: automated-test workflow; physical OS unverified.



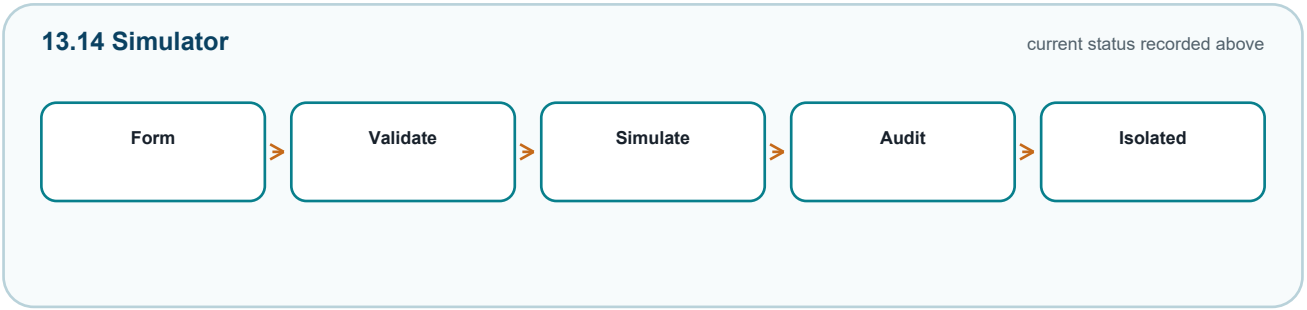
13.13 Restoration

Resolved/reviewed hold → Main Admin fresh MFA → ALLOW command → agent validates/execut es → ALLOWED acknowledgment → ACTIVE/session restoration. Status: automated-test workflow; physical OS unverified.



13.14 Simulator

Admin form → bounded synthetic route → read-only evaluation → simulation-only result + audit history → no employee/ML/command mutation. Status: simulated and automated-test verified.



14. Testing, deployment, limitations and future scope

The current status is evidence-based and includes fresh validation results.

Command	Fresh result on 4 Nov 2026
node --test --test-concurrency=1 server/test/*.test.js	139 total: 137 passed, 0 failed, 2 skipped; skips are optional DB-IP tests
node --test --test-concurrency=1 agent/test/*.test.js	140 total: 130 passed, 2 failed, 8 skipped; two failures in agent/test/core.test.js
python -B -m unittest discover -s ml/test -v	29 passed, 0 failed; plumbing/contract/research tests
npm run build	Passed; Prisma generated, 2,305 Vite modules transformed; chunk-size advisory
Physical acceptance	Not performed: Windows two-account/USB/reboot and macOS native/runtime acceptance

14.1 Current failures and skips

The two current agent failures are `enrollment expiry/reuse...` because a mocked transaction lacks `endpointEvent.findMany`, and `all three platforms get fresh account instructions...` because an endpoint-assignment fixture is rejected. These are not silently reclassified as passes. Eight agent skips are platform/release prerequisites. The backend and ML results are current fresh results; older audit totals must not replace them.

14.2 Deployment guide

Install Node/Python/PostgreSQL, configure DATABASE_URL/JWT secret/origins/Mailjet as appropriate, run npm ci, Prisma generate/migrate, build and start. Run ML service separately. For endpoint updates preserve enrollment/queue/config and use installed updater. Production needs HTTPS, trusted proxy, secrets management, package signing/release acceptance and managed OS permissions.

14.3 Limitations and future scope

Verified limitations are 0/21 endpoint CERT compatibility, no promoted endpoint-native RF, uncalibrated RF score, partial platform telemetry, no physical Windows containment acceptance, no validated Mac account-wide enforcement, conservative USB attribution, in-app rather than external security notifications, disabled employee console, in-process SSE and mutable PostgreSQL audit storage. Future work is real privacy-reviewed endpoint data/labels, endpoint-native model review, physical Windows/Mac acceptance, managed Mac enforcement, enterprise identity/resource integration, external notifications, distributed processing and tamper-evident archival.

15. Glossary and two-minute revision

These definitions connect academic concepts to the actual project.

Term	CYBERORBIT meaning
UEBA	User and Entity Behavior Analytics over identities/devices and historical context
Zero Trust	Explicit continuous verification of identity, device, resource and context
Feature	Numeric model input; active RF contract has 21 user-day features
Inference	Using the loaded saved model to produce an RF score
Provenance	Evidence source, identity, timing, schema and trust context
Fail closed	Withhold success when proof is missing or invalid
SSE	One-way server-to-browser event stream
MFA	OTP/step-up verification for admin and sensitive operations
Containment	Application restriction or acknowledged endpoint OS action
Idempotency	Repeated IDs do not duplicate an effect
SHAP	Tree-model feature-contribution explanation for complete aggregate inference

15.1 Architecture in two minutes

Browser → Express/auth → scoped security services → Prisma/PostgreSQL, with agents delivering authenticated telemetry and FastAPI serving guarded RF inference. All authority remains server-side.

15.2 ML in two minutes

The saved RF uses an exact CERT user-day contract. It is loaded and validated, not trained live. Endpoint data is not compatible for 21/21 verified features, so live endpoint RF is unavailable. Complete aggregates can receive SHAP reasons; the score is uncalibrated.

15.3 Cybersecurity in two minutes

MFA sessions, roles, company/assignment scopes, device identity, trust, resource sensitivity, contextual factors, policy bands, audits, deduplicated notifications and signed expiring commands form the Zero Trust control plane. Containment remains fail-closed.

15.4 Full pipeline in two minutes

Enroll → heartbeat/observe → encrypt/ingest → validate/persist → assess context and optional model → policy/trust gates → allow, monitor, MFA, approval or restriction → incident/audit/notification → optional signed BLOCK/ALLOW and native acknowledgment.

Appendix: evidence paths

Primary evidence paths include package.json; client/src/App.jsx and client/src/pages; server/server.js; server/routes/index.js and mounted route modules; server/middleware/auth.js; server/services/security-service.js, endpoint-service.js, risk-analysis-service.js, policy-service.js, ml-service.js, sse.js and ueba-service.js; cyber/risk-engine/risk-engine.js and cyber/zero-trust/trust-engine.js; prisma/schema.prisma and migrations; ml/src/* and ml/models/*; agent/src/*, agent/native/*, agent/packaging/* and agent/README.md; server/test/*, agent/test/* and ml/test/*.

The separate viva pages from the previously generated source-grounded question bank are appended after this body so the final artifact remains one PDF and contains exactly 100 questions with answers and follow-ups.

25. Extended technical viva question bank

This section contains 100 implementation-grounded questions in the required 10/10/20/20/10/10/5/5/5/5 distribution. Each item includes a defensible answer and a likely follow-up.

A. Project introduction and objectives

1. What is CYBERORBIT?

Answer: It is an administrator security console that combines endpoint telemetry, contextual risk, Zero Trust policy enforcement, UEBA-style baselines, guarded Random Forest inference, incidents, notifications and endpoint command workflows. The implementation is in React/Vite, Express/Prisma/PostgreSQL, Python FastAPI and a separate JavaScript agent.

Follow-up: Which component makes the final access decision? The backend security and policy services, not the browser.

2. Why was CYBERORBIT developed?

Answer: The project addresses the gap between static access control and continuous, context-aware monitoring. A valid identity can still use an unusual device, time, location, volume or resource, so the backend evaluates current evidence before protected access.

Follow-up: Does every unusual observation prove an attack? No; it is evidence requiring context or review.

3. What is the project’s main objective?

Answer: To demonstrate a controlled Zero Trust/UEBA platform that collects endpoint observations, preserves provenance, evaluates policy risk, explains decisions and supports review/containment workflows without converting missing evidence into a benign answer.

Follow-up: What is the strongest limitation? The CERT model is not live endpoint-native ML.

4. Who are the main users?

Answer: Security and system administrators are the primary console users. Manager assignment and investigation concepts exist, while employee console login is deliberately disabled in the current API.

Follow-up: How are employee decisions exercised? Through server-side evaluation and tests rather than an employee production session.

5. What is the difference from traditional monitoring?

Answer: Traditional monitoring often reports events after the fact. CYBERORBIT combines current identity, device, resource, trust, historical baseline and risk-policy context before a protected-resource decision.

Follow-up: Is the system an enterprise SIEM replacement? No; it is a scoped capstone control plane.

6. What is the project’s actual innovation?

Answer: The defensible contribution is the integration of provenance-aware endpoint evidence, contextual risk, guarded legacy ML, Zero Trust gates, explainable factors, scoped notifications and fail-closed containment in one demonstrable system.

Follow-up: What should not be claimed? Autonomous guaranteed insider-threat detection.

7. Why are endpoint observations important?

Answer: They provide evidence close to the host: process/application/session state, USB inventory, file changes, network metadata and health. Their meaning is platform-specific and they are not automatically proof of user intent.

Follow-up: Why can device-wide events be risky to attribute? They may not map to a specific employee.

8. What is an insider threat?

Answer: It is potential harm involving an internal identity or trusted access. CYBERORBIT treats indicators as risk evidence and does not label an employee malicious from a single observation.

Follow-up: Give an example indicator: unusual volume plus a privilege or transfer observation.

9. What is the project scope?

Answer: It covers administrator authentication, monitoring identities/devices, endpoint telemetry, risk/policy/incident workflows, model integration, simulation, notifications and command acknowledgments. Enterprise SSO, external resource gateways and immutable audit archival are outside scope.

Follow-up: Which scope item is only simulated? Simulator scenarios and custom synthetic inputs.

10. What is the project's overall status?

Answer: It is suitable for a controlled prototype demonstration of tested backend/browser/simulator and agent contracts. Physical endpoint acceptance, endpoint-native ML and some production integrations remain incomplete.

Follow-up: What fresh evidence supports that? Backend 137/139 pass, ML 29/29 pass, agent 130 pass/2 fail/8 skip and build passed.

B. System architecture

11. Describe the high-level architecture.

Answer: The React/Vite browser calls an Express API. Express authenticates and scopes requests, services call Prisma/PostgreSQL, endpoint agents call dedicated endpoint routes, and the Node ML client calls FastAPI Python for complete user-day aggregates. SSE and polling update the console.

Follow-up: Where is the system of record? PostgreSQL.

12. What is the server entry point?

Answer: `server/server.js` loads environment configuration, creates Express, installs security/CORS/body handling, mounts `/api`, starts Vite development middleware or production static hosting, and schedules retention/snapshot tasks.

Follow-up: What happens on duplicate startup? The listener is reserved before Vite and the server fails clearly.

13. Why is there a separate Python service?

Answer: Python owns model loading, exact dataframe validation, scikit-learn inference and SHAP TreeExplainer. Node remains the authority for authentication, company policy, persistence and risk integration.

Follow-up: Can Python authorize an account by itself? No; the response says inference only.

14. How does an endpoint communicate?

Answer: It exchanges a one-use enrollment token for a device credential, then sends authenticated heartbeats and versioned event batches. The backend validates device/company/assignment and persists accepted events.

Follow-up: What protects event retries? Device/event uniqueness and acknowledgment.

15. How does the frontend communicate?

Answer: Client service helpers use JSON fetches with bearer tokens and refresh handling. `App.jsx` also creates an SSE stream for scoped events and polls persisted notifications every 60 seconds while visible.

Follow-up: Is every dashboard value SSE-driven? No; most pages fetch data on mount or refresh.

16. Which component owns authorization?

Answer: The backend middleware and security services. Browser visibility, client-supplied roles and client risk scores are not authoritative.

Follow-up: What does the protected resource route derive? The principal from the server-validated session.

17. How are incidents generated?

Answer: Qualifying risk/policy decisions create or update persisted incidents with severity, factors, explanation, recommended action and user scope. Duplicate evidence is guarded by event/incident logic.

Follow-up: Does ordinary process activity automatically create an incident? No.

18. How are notifications delivered?

Answer: SecurityNotification rows are persisted with recipient/dedupe keys. Admin APIs provide list/unread/read operations; SSE pushes scoped updates and visible-tab polling recovers persisted alerts.

Follow-up: Are they email alerts? No, the bell is in-app; Mailjet is for OTP/reset email.

19. What is the deployment shape?

Answer: A trusted HTTPS proxy fronts Node, PostgreSQL stores durable data, the built SPA is served by Node in production, and FastAPI runs as a separate internal service. Agents connect through the trusted endpoint URL.

Follow-up: What is still deployment responsibility? HTTPS, secrets, signing, proxy and database operations.

20. How does the system fail safely?

Answer: Invalid auth, stale sessions, invalid endpoint credentials, incomplete ML data, schema mismatches, unsupported native results and expired/replayed commands return bounded errors or unavailable states rather than success.

Follow-up: What does missing data mean? Unavailable coverage, not benign behavior.

C. Machine Learning and AI

21. What is artificial intelligence?

Answer: AI is the broad field of systems performing tasks associated with reasoning or decision support. In CYBERORBIT, most security decisions are explicit rules/policy; ML is one guarded evidence source and Gemini is optional.

Follow-up: Why not call every rule AI? Because its implementation is deterministic policy code.

22. What is supervised learning?

Answer: A supervised model learns from examples with labels. The preserved CERT artifact is described as a Random Forest classifier trained on labeled user-day threat examples, but original raw data and answer key are absent.

Follow-up: Can the saved metrics be reproduced here? Not independently from the checkout.

23. What is unsupervised learning?

Answer: It finds structure without labels, often for anomaly detection. Isolation Forest is discussed in historical/legacy fields and theory, but no live Isolation Forest inference path is confirmed.

Follow-up: What is the deployed active model? RandomForestClassifier.

24. What is classification?

Answer: Classification assigns an observation to a class. The RF returns a positive-class score and compares it with threshold 0.85 for a predicted class.

Follow-up: Does that score equal attack probability? No, calibration is not verified.

25. What is a feature?

Answer: A feature is a numeric model input. The CERT contract contains 21 user-day count, distinct-count, calendar and sum features in exact schema order.

Follow-up: Why is schema order important? Node and Python validate the same named ordered contract.

26. What is a label?

Answer: A label is the outcome the supervised model learns, such as a threat/non-threat class. Current endpoint research requires independently reviewed labels; simulator outcomes and risk scores cannot be reused as labels.

Follow-up: Why avoid using freeze as a label? It would create circular policy leakage.

27. What is training?

Answer: Training fits model parameters using historical examples. The checked-in artifact is loaded, not retrained by the production backend; experimental training is isolated under `ml/research`.

Follow-up: Does endpoint telemetry retrain the live model? No.

28. What is validation?

Answer: Validation measures choices such as threshold on data separate from training. The saved notebook records a chronological 70/15/15 row split; research scripts support stricter split strategies.

Follow-up: Is the original split employee-disjoint? The audit says no.

29. What is testing in ML?

Answer: Testing evaluates a held-out set. Saved metadata reports test metrics, but the original data is missing, so they are historical saved claims rather than freshly reproduced results.

Follow-up: What metric should be considered carefully? Recall is only 0.2416 in saved metadata.

30. What is overfitting?

Answer: Overfitting is learning noise or leakage instead of general patterns. CYBERORBIT's research pipeline checks provenance, split overlap and suspicious features but cannot certify absent source semantics.

Follow-up: What future split is safer? Time-forward and preferably employee-disjoint.

31. What is data leakage?

Answer: Leakage occurs when training uses information unavailable at decision time or influenced by the outcome. The endpoint research pipeline checks availability times and rejects post-event/target fields.

Follow-up: Can a manually falsified availability timestamp be detected fully? No.

32. What is class imbalance?

Answer: A rare positive class makes accuracy misleading. The saved test set has 149 threats versus nearly 50,000 rows; precision/recall/F1 are more informative than accuracy alone.

Follow-up: What is the danger of high accuracy here? Predicting mostly negative can still miss threats.

33. How does Random Forest work?

Answer: It trains many decision trees on bootstrap samples, randomizes candidate features at splits, and aggregates their votes/probabilities. This reduces variance compared with one tree.

Follow-up: What are the recorded parameters? 150 trees, max depth 14, min split 6, min leaf 2, balanced classes, seed 42.

34. What is the RF output in CYBERORBIT?

Answer: Python calls `predict_proba` for class 1, returns the score and thresholded class, then returns model/schema fingerprints and aggregate provenance. Node checks all of these before integration.

Follow-up: What happens on mismatch? The prediction is rejected fail-closed.

35. What is Isolation Forest?

Answer: Isolation Forest randomly partitions points; unusual points tend to be isolated in fewer splits and receive higher anomaly scores. That theory is not a confirmed live CYBERORBIT decision path.

Follow-up: Should you claim it scores endpoint risk? No.

36. What is SHAP?

Answer: SHAP estimates feature contribution to a model output. `ml/src/xai.py` uses TreeExplainer and returns the top five contributions for complete RF aggregates.

Follow-up: Are contextual incident explanations always SHAP? No, they may be stored policy-factor text.

37. List the 21 features.

Answer: They are logon_count, device_event_count, file_event_count, http_event_count, email_event_count, day_of_week, is_weekend, five unique-PC counts, unique_files, file_content_events, unique_urls, unique_http_content, unique_recipients, email_size_total, attachments_total, total_activity and cross_source_activity.

Follow-up: What is their aggregation level? User-day.

38. Why is live CERT inference unavailable?

Answer: Endpoint events do not provide verified equivalents for the five source families and their semantics; the repository audit rates live compatibility 0/21. Missing features are not silently zero-filled.

Follow-up: What is accepted by `/api/ml/predict`? A complete externally supplied aggregate.

39. What is endpoint-native ML readiness?

Answer: It requires a versioned feature contract, real representative telemetry, stable identity/time windows, independently adjudicated labels, leakage review, calibration and held-out evaluation. No such promoted model exists.

Follow-up: Are synthetic experiments performance evidence? No, they validate plumbing only.

40. What is the ML data pipeline?

Answer: The preserved pipeline is source aggregation → exact contract → saved RF training/artifact → Python load → strict inference → SHAP reasons → Node threshold mapping → risk/policy. Endpoint telemetry currently stops at readiness diagnostics.

Follow-up: Which stage is future for endpoints? Compatible labeled aggregation and training.

D. Cybersecurity and Zero Trust

41. What is Zero Trust?

Answer: Zero Trust means no implicit trust: continuously verify identity, device, resource, authorization and current context. CYBERORBIT applies MFA sessions, scoping, trust, risk and policy gates.

Follow-up: What is not implemented? Enterprise SSO/hardware MFA/resource gateway.

42. What is authentication?

Answer: Authentication verifies who is making a request. CYBERORBIT uses password verification, OTP/MFA, signed JWTs, persisted sessions and endpoint credentials.

Follow-up: What is authorization? It decides what the verified principal may do.

43. What is RBAC?

Answer: Role-based access control assigns permissions through roles. CYBERORBIT has EMPLOYEE, MANAGER, SECURITY_ADMIN and SYSTEM_ADMIN plus resource required roles.

Follow-up: Can approval grant an absent role? No.

44. What is least privilege?

Answer: Give only the access required. Company, assigned-admin, manager, resource-department and required-role checks limit data and action scope.

Follow-up: Who has broadest organization view? Main Admin within the company.

45. What is device trust?

Answer: It is a posture signal about a registered logical or endpoint device. CYBERORBIT uses trust score/status, enrollment, mapping, revocation and heartbeat freshness.

Follow-up: Does registration alone prove measured trust? No; unassessed defaults are hidden.

46. What is UEBA?

Answer: User and Entity Behavior Analytics learns or compares behavior for people and devices. CYBERORBIT's baseline uses prior daily volume, login hours and locations, while endpoint events add coverage.

Follow-up: What is the baseline window? Prior 30 days, excluding current day, requiring seven days.

47. What is a risk-based access decision?

Answer: The backend evaluates current signals and maps a bounded score to ALLOW, MONITOR, MFA, approval, restriction or freeze according to tenant policy.

Follow-up: Who performs it? Server-side services, not React.

48. State the exact risk formula.

Answer: Contextual evidence is the sum of the strongest factor in each category, capped at 100. RF score is piecewise mapped around its operating threshold. Final risk is rounded max(ML evidence, contextual evidence), bounded 0–100.

Follow-up: Why max instead of weighted sum? To avoid double-counting overlapping evidence.

49. What are the default action bands?

Answer: The documented defaults are 0–39 ALLOW, 40–59 MONITOR, 60–74 MFA, 75–89 reviewed approval, and 90–100 FREEZE if enabled. Stored company policy can customize boundaries.

Follow-up: What must you check for a live tenant? The stored policy.

50. Is the 70/30 formula active?

Answer: No. It exists in legacy `ml/src/risk.py`; the authoritative backend uses max of threshold-mapped ML and contextual evidence.

Follow-up: How should this be defended in viva? Cite `cyber/risk-engine/risk-engine.js`.

51. Why is off-hours not enough?

Answer: The engine adds off-hours only when another factor already exists; timing is corroborating context rather than standalone malicious evidence.

Follow-up: What other corroboration might exist? Volume, privilege, auth failure or untrusted device.

52. What is an alert?

Answer: An alert is a notification-worthy observation or policy outcome. It must have evidence, identity/tenant scope, classification and deduplication; not every event becomes an alert.

Follow-up: What is the USB rule? Connection does not prove transfer.

53. What is incident management?

Answer: Incidents persist user, severity, status, risk, factors, explanation, recommended action and resolution information for administrator review.

Follow-up: Can a browser-only alert prove physical containment? No.

54. How is audit logging used?

Answer: Policy, authentication, access, admin, simulation and containment actions write application audit records with company/actor/time/category/details.

Follow-up: Is the database immutable? No; DB administrators can alter storage.

55. What is replay protection?

Answer: A command or token must not be accepted twice. CYBERORBIT uses unique IDs, expiry, device/action/identity binding and durable agent journals.

Follow-up: What happens on replay? It is rejected or returns the persisted result without repeating OS work.

56. What is signed endpoint command security?

Answer: Commands are authenticated with device-derived keys/signatures and checked for exact shape, device, mapped identity, action, issue time, expiry and replay state.

Follow-up: Does a valid signature alone prove OS success? No; native acknowledgment is required.

57. Why is containment fail-closed?

Answer: Disabling/restoring an account is high impact. The system requires fresh evidence, supported mapping, authorization, command integrity and native acknowledgment; uncertainty stays pending or failed.

Follow-up: What does unsupported macOS employee mode do? It refuses account-wide control.

58. Why is USB attribution conservative?

Answer: A connected device is observed at endpoint scope and may not identify a person or transfer. Current alerts use device-level wording, null employee target and dedupe state.

Follow-up: What would prove transfer? An explicit validated transfer observation, not connection alone.

59. What is MFA?

Answer: Multi-factor authentication combines more than one verification factor. CYBERORBIT uses OTP flows for admin login and fresh step-up for sensitive operations.

Follow-up: Which action needs Main Admin fresh MFA? Restoration/unfreeze paths.

60. What is the difference between policy and evidence?

Answer: Evidence is an observed or supplied signal; policy maps evidence and context to an action. A policy decision is not itself proof that a threat occurred.

Follow-up: Why store provenance? To let reviewers distinguish observation, inference, simulation and decision.

E. Frontend and React

61. How is the React application organized?

Answer: `App.jsx` is the authenticated console shell; pages under `client/src/pages` implement workflows, reusable components live under `components`, and services handle API/session helpers.

Follow-up: Is React Router declared? The current shell uses active page/tab state rather than a router dependency.

62. How is authentication state managed?

Answer: App state plus `zero_trust_token` and session helper logic load `/api/auth/me`, attach bearer tokens, refresh sessions and clear state on logout/failure.

Follow-up: Where is refresh protection? HTTP-only cookie plus CSRF/origin checks.

63. What does DashboardPage display?

Answer: It requests summary, risk-distribution and risk-trend data and renders cards/charts/tables using backend responses. It does not independently calculate authoritative risk.

Follow-up: What file is currently user-modified? `client/src/pages/DashboardPage.jsx`; it was preserved.

64. How does the notification bell work?

Answer: App fetches persisted notifications on admin login/visibility and every 60 seconds, subscribes to SSE, deduplicates IDs and calls read/read-all endpoints.

Follow-up: What if SSE fails? Polling/persisted data remains the fallback.

65. How are charts populated?

Answer: Page components call scoped APIs and transform response arrays into Recharts-compatible series. Empty, loading and error states are retained rather than fabricated.

Follow-up: Are chart values a second risk engine? No.

66. What does the Devices page do?

Answer: It lists logical devices and endpoint state, supports assignment/configuration/action controls and displays endpoint monitor feedback from API routes.

Follow-up: What does token issuance not prove? Physical enrollment.

67. What does Explainable AI show?

Answer: It displays recorded factor explanations and, where available, model/SHAP-derived information from the approved analysis path. Stored policy text is labelled advisory.

Follow-up: Does every event get SHAP? No.

68. How are errors handled in the UI?

Answer: Pages use local loading/error state, bounded API helpers, cancellation where needed, visible feedback and empty states. A failure does not fabricate an alert or risk score.

Follow-up: Why is this important in security UI? False certainty can cause unsafe action.

69. What does the Simulator page do?

Answer: It loads scenarios/history and submits bounded synthetic context to dedicated simulation routes. Results are labelled synthetic and isolated from production employee history.

Follow-up: Can it issue a real BLOCK? No, tests assert it cannot.

70. What are the frontend security restrictions?

Answer: The UI is not the authorization boundary. Backend role/MFA/company/assignment checks enforce access even if a client is modified.

Follow-up: What should a faculty member inspect? Route middleware and server scoping.

F. Backend and APIs

71. What does Express do?

Answer: It receives HTTP requests, applies middleware and routes them to services, and serves Vite/static frontend content depending on environment.

Follow-up: What does Express not own? Model training or database authority by itself.

72. What does auth middleware verify?

Answer: JWT signature/algorithm, expiry, subject, MFA, user existence/status, auth version/password changes and persisted session validity.

Follow-up: What happens to an employee console token? Current middleware rejects non-admin console roles.

73. What are services?

Answer: They contain domain logic such as security assessment, endpoint workflows, policy persistence, ML integration, sessions, notifications, SSE and baselines, keeping routes comparatively thin.

Follow-up: Why use transactions? To make correlated security records consistent.

74. How does input validation work?

Answer: Express applies bounded parsers; route/service code validates fields, enums, ranges, IDs, event contracts, model invariants and command state.

Follow-up: What happens to invalid model data? It is rejected before inference/persistence.

75. How does error handling work?

Answer: Structured SecurityError/API wrappers and endpoint error middleware return codes/statuses without secrets or raw provider/native details.

Follow-up: Why keep machine-readable codes? UI and agents can distinguish expiry, scope, unsupported and replay.

76. What is the ML API contract?

Answer: Node sends a complete named 21-feature aggregate to Python `/predict`; Python returns score, class, threshold, fingerprints, aggregation level, provenance and XAI reasons; Node verifies the response.

Follow-up: Can a single endpoint event be expanded? No.

77. What is the protected resource path?

Answer: Session-authenticated route derives the principal, loads the company resource, calls security service/risk/policy/authorization, persists evidence and returns data only when allowed.

Follow-up: Can the client supply a role override? No.

78. What are the endpoint API groups?

Answer: Assignment, enrollment token, enroll, heartbeat, ingest, dashboard, config, city estimate, revoke, stop/start agent, USB policy and local-operation request/review/status/completion.

Follow-up: What authenticates agent calls? Endpoint credential, not browser JWT.

79. What is SSE in the backend?

Answer: `SseManager` keeps authenticated connections, sends heartbeat, rechecks authorization/company/assignment and broadcasts scoped activity/security events.

Follow-up: Is it distributed? No, it is in-process with persistence/polling fallback.

80. What background tasks exist?

Answer: Endpoint retention and periodic daily-risk-snapshot refresh run as bounded application timers; SSE maintains a heartbeat timer.

Follow-up: Is there a distributed job queue? Not in the checked-in implementation.

G. Database and PostgreSQL

81. Why PostgreSQL?

Answer: It provides durable relational storage, transactions, indexes, unique constraints and JSON fields for users, endpoint events, security workflows and histories.

Follow-up: How does Prisma use it? Generated client/schema/migrations and transactional queries.

82. What is Prisma?

Answer: Prisma is the ORM and schema/migration toolkit. `schema.prisma` defines models, enums, relations, indexes and constraints.

Follow-up: Does Prisma replace authorization? No; services still scope every query.

83. How are endpoint events deduplicated?

Answer: `EndpointEvent` has a unique `(deviceId,eventId)` constraint and service logic inserts only new accepted IDs.

Follow-up: Why does receivedAt differ from occurredAt? It distinguishes observation time from server arrival.

84. How is tenant isolation stored?

Answer: Company IDs appear on users and operational entities; relation/scoping filters and unique policy/notification rules keep records within company/recipient scope.

Follow-up: Is tenant isolation only a UI filter? No, route/service/database queries enforce it.

85. What stores containment history?

Answer: EndpointEnforcementCommand records action/status/reason/expiry/acknowledgment/device/user; User status/session/audit/incident records preserve application workflow.

Follow-up: What is not stored as proof? An unacknowledged OS outcome.

H. Windows/macOS endpoint agents

86. How does Windows enrollment work?

Answer: Admin assignment records a verified Windows SID and issues a short-lived one-use token. The installer exchanges it, stores protected credential/queue state and begins heartbeat.

Follow-up: Should the token be placed on the command line? No, hidden input is used.

87. How does macOS differ?

Answer: macOS employee mode uses a per-user LaunchAgent, Keychain/config and user-scoped collectors; native permissions and system scope differ. A user-mode agent cannot enforce an account-wide block.

Follow-up: What is the Mac build kit? A builder handoff, not necessarily an employee package.

88. How does the queue help?

Answer: Events are encrypted and durably queued locally; uploader retries and removes only acknowledged IDs, supporting offline operation.

Follow-up: Does offline data authorize a new freeze? No, fresh authenticated heartbeat/evidence is required.

89. What does a heartbeat carry?

Answer: Health, lifecycle/config state, account/USB/agent command reports and receives fresh configuration/commands. It also establishes endpoint freshness.

Follow-up: What if it is revoked? The agent stops its authenticated path.

90. What is physically unverified?

Answer: Real Windows two-account disable/reboot/recovery and USB insertion, plus macOS launchd/Keychain/TCC/native helper/runtime acceptance. Source tests are not physical acceptance.

Follow-up: How should this be stated in viva? Automated-test verified; physical behavior not verified.

I. Risk, incidents and containment

91. What is the difference between BLOCK and ALLOW?

Answer: BLOCK is a containment instruction intended to restrict the mapped endpoint account; ALLOW is restoration. Neither changes the OS merely because the command exists; native acknowledgment is required.

Follow-up: What reports success? Explicit `BLOCKED` or `ALLOWED` result, subject to workflow state.

92. Why require Main Admin restoration?

Answer: Restoration reverses a high-impact restriction, so the workflow requires critical-review state, Main Admin authorization and fresh MFA before a signed ALLOW command.

Follow-up: Can an old approval override a missing role? No.

93. What makes automatic containment eligible?

Answer: Explicit opt-in, fresh authenticated protected-resource evidence, freeze-level contextual risk, independent corroborating categories, current supported endpoint mapping and policy requirements.

Follow-up: Does RF availability alone authorize it? No.

94. How are incidents and notifications related?

Answer: An incident is the investigation record; a SecurityNotification is a recipient-scoped in-app delivery row. A notification can point to an incident and is deduplicated separately.

Follow-up: What is the UI fallback? Persisted polling.

95. Why is containment fail-closed?

Answer: An incorrect block can harm a real employee and an incorrect restore can remove protection. Uncertainty remains pending/unsupported/failed until proof arrives.

Follow-up: What is the final limitation? Physical OS acceptance remains required.

J. Testing, limitations and future scope

96. What are the latest backend results?

Answer: Fresh execution produced 139 tests: 137 passed, 0 failed and 2 skipped for optional DB-IP lookups.

Follow-up: Does this prove production deployment? No.

97. What are the latest agent results?

Answer: Fresh execution produced 140 tests: 130 passed, 2 failed and 8 skipped. The two failures are current `agent/test/core.test.js` fixture/compatibility failures.

Follow-up: What should happen before release? Diagnose and resolve or explicitly document the fixture contract.

98. What are the ML test results?

Answer: All 29 Python ML tests passed. They validate inference, schema, SHAP, staging, provenance and synthetic research plumbing, not endpoint model performance.

Follow-up: Why not call them accuracy tests? There is no endpoint-native labeled evaluation set.

99. What limitations should be presented?

Answer: 0/21 verified endpoint CERT features, no promoted endpoint RF, uncalibrated saved RF score, partial platform telemetry, pending physical containment, unvalidated Mac employee package/runtime and no immutable audit archive.

Follow-up: What is future scope? Real labeled endpoint data, managed Mac enforcement, physical acceptance and production hardening.

100. How should the project be defended honestly?

Answer: Describe what source/tests prove, label synthetic and unavailable paths, cite the live max-risk formula, and state that containment/ML readiness are bounded prototype capabilities rather than universal guarantees.

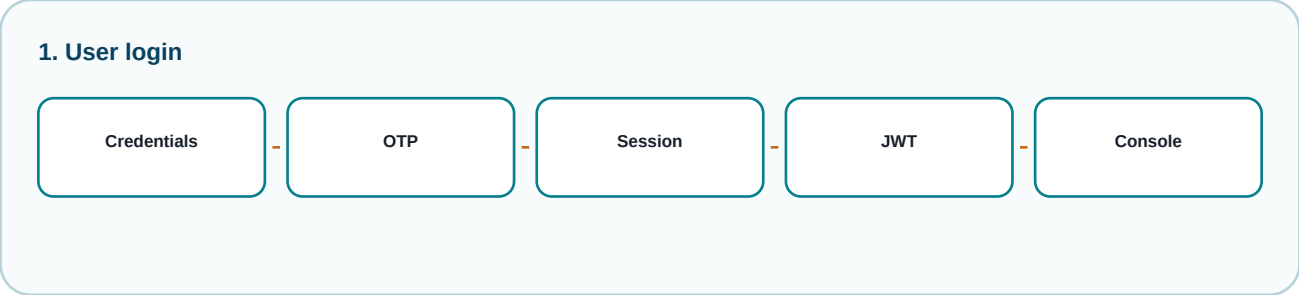
Follow-up: What is the best evidence strategy? Show source path, test name and status for every claim.

Pipeline supplement: 19 explicitly enumerated workflows

This supplement expands the pipeline chapter into the exact 19 workflow names requested. Each entry identifies the trigger, principal data, validation/decision boundary and implementation status. The source is the current repository, not a theoretical replacement.

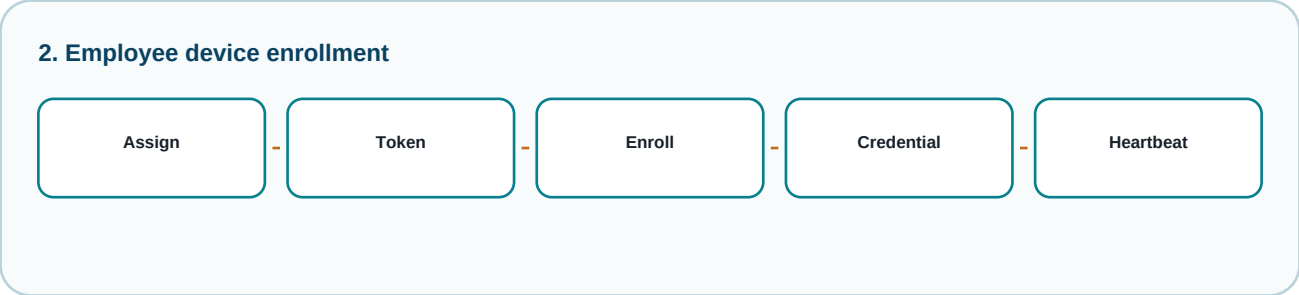
1. User login

Browser submits admin credentials; auth-service validates password; MFA challenge/verification creates a persisted session and short-lived JWT; App loads /auth/me. Status: automated-test verified.



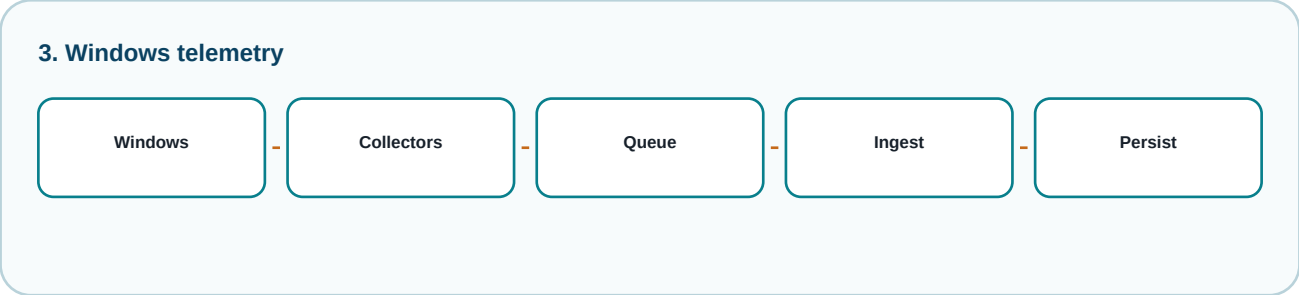
2. Employee device enrollment

Admin assignment records company/user/platform/SID or UID; expiring one-use token is exchanged by the agent; credential hash/config/device row persist. Status: automated-test verified; physical install unverified.



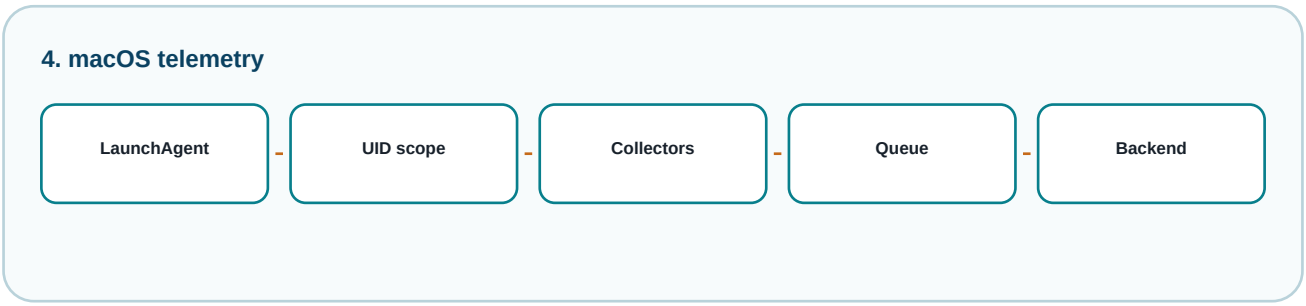
3. Windows telemetry

Windows collectors/native helpers normalize process, session, USB, network, file/application and health observations; encrypted queue/uploader sends endpoint events. Status: contract tested; physical coverage partial.



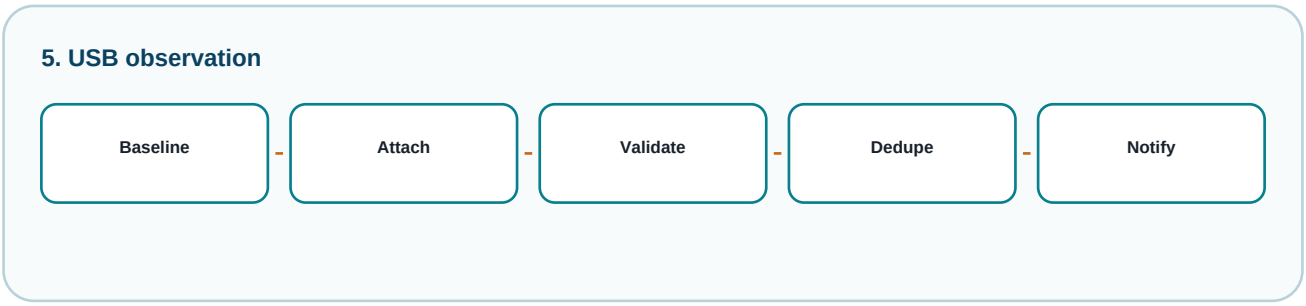
4. macOS telemetry

LaunchAgent user-mode collectors use UID-scoped process/session/network/system_profiler USB and desktop helpers; Keychain/queue protect state; heartbeat/ingest persist events. Status: source/automated contract; native Mac runtime unverified.



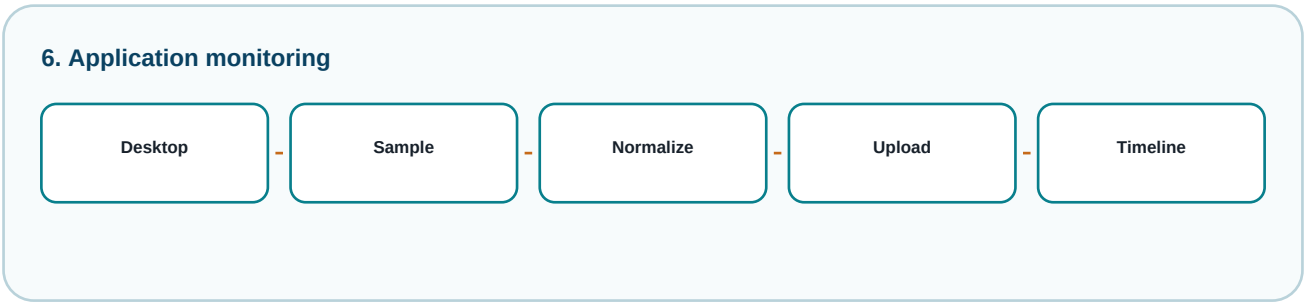
5. USB observation

Baseline is quiet; a later hardware change emits USB_CONNECTED; credential, assignment, event ID and lifecycle state are checked; device-level row/notification is deduplicated. Status: automated-test verified; physical insertion unverified.



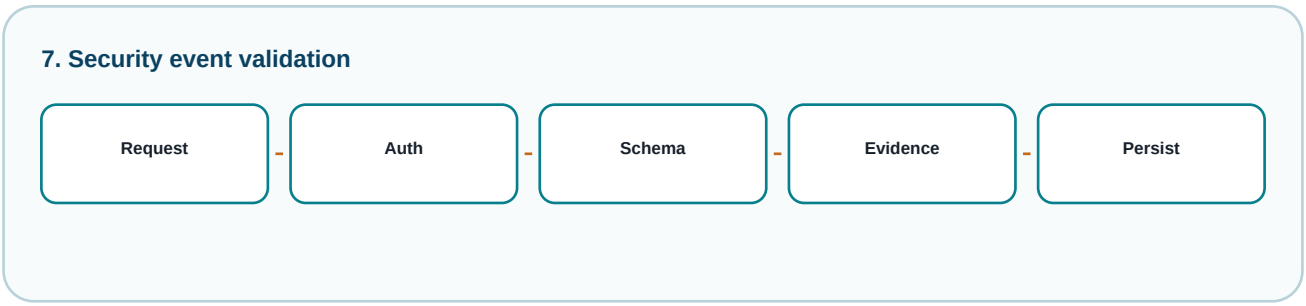
6. Application monitoring

Desktop sampling detects eligible foreground transitions and usage duration; event normalization preserves app/process identity; uploader and endpoint dashboard display records. Status: automated-test verified; brief activity can be missed.



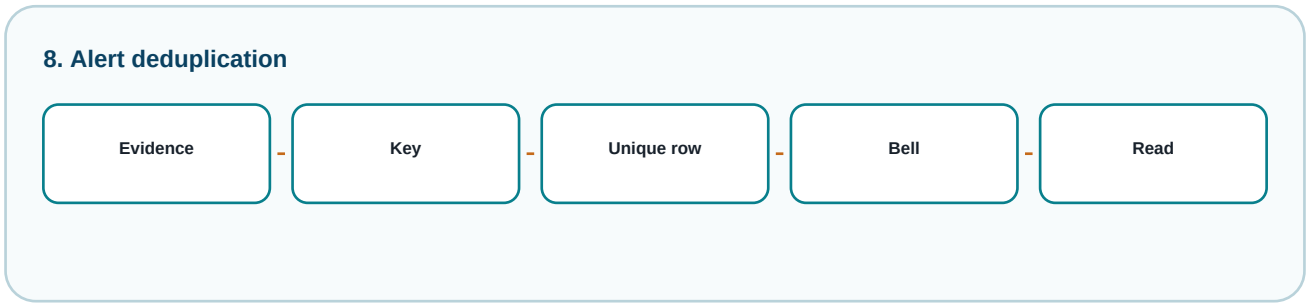
7. Security event validation

Request/event reaches middleware and service; auth, company, endpoint assignment, schema, time, event ID and evidence semantics are checked before persistence or rejection. Status: automated-test verified.



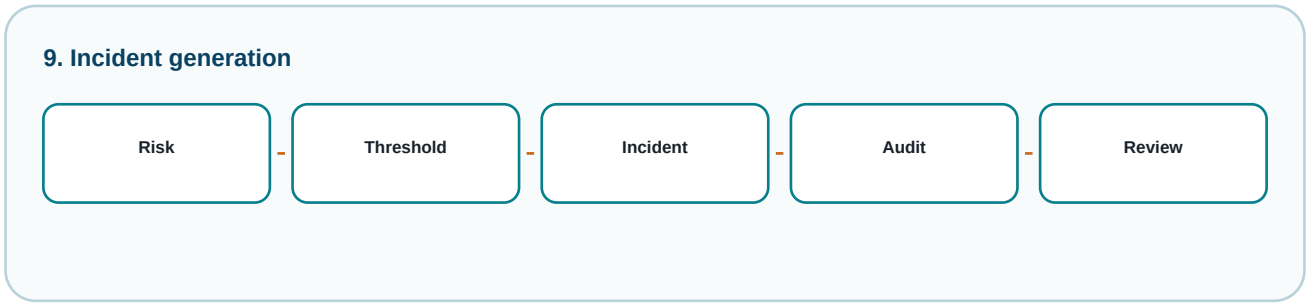
8. Alert deduplication

Incident/event gets a stable dedupe key; database uniqueness and accepted-event IDs prevent duplicate rows; client merges by notification ID. Status: automated-test verified.



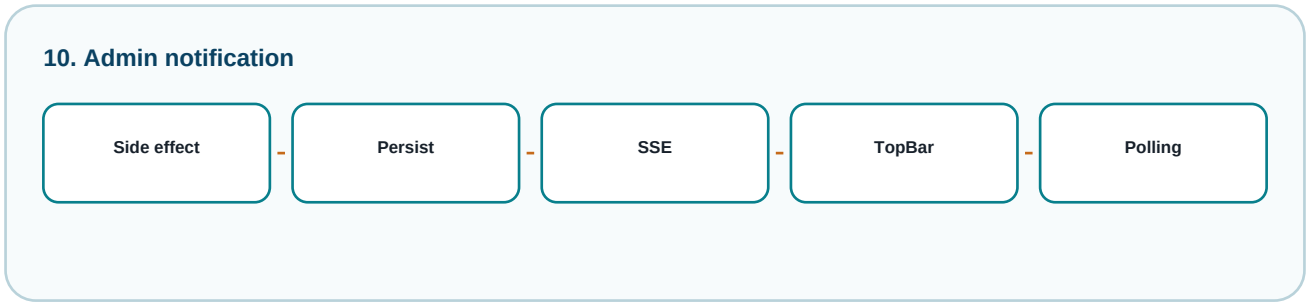
9. Incident generation

Qualifying score/action passes incident threshold and scope checks; Incident stores severity, factors, explanation and recommended action; audit/notification side effects follow. Status: automated-test verified.



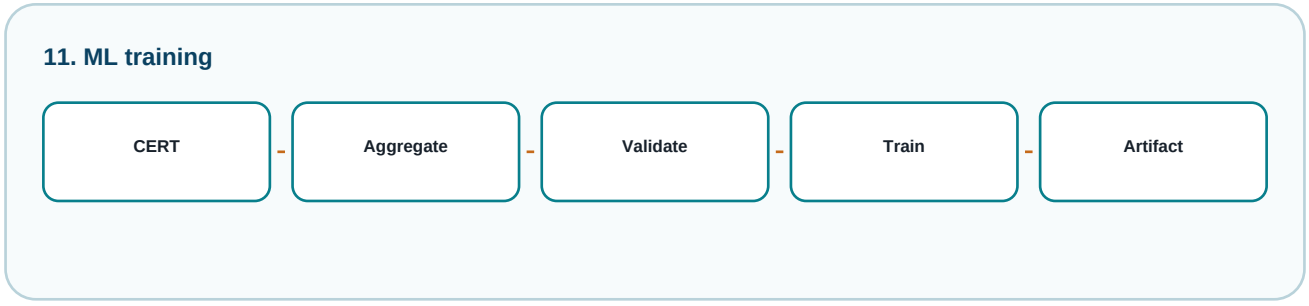
10. Admin notification

SecurityNotification is recipient/company scoped and persisted; SSE sends matching recipient event; TopBar updates; visible-tab polling recovers persisted state. Status: automated/browser fixture verified; production delivery unverified.



11. ML training

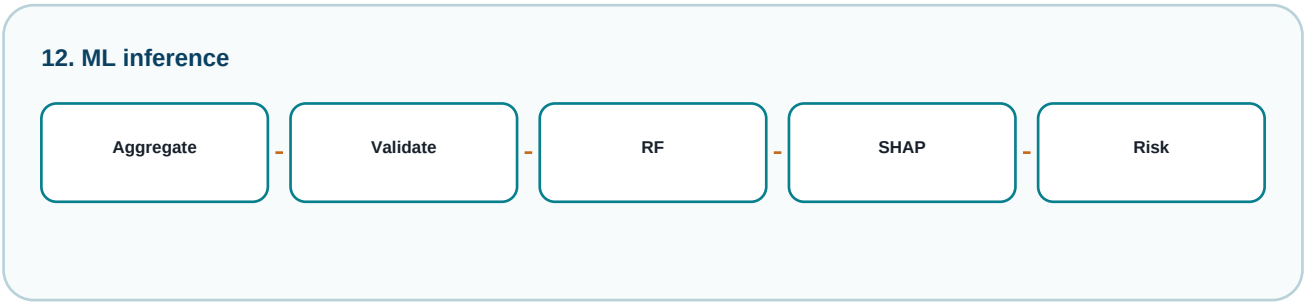
CERT source families become user-day rows; notebook/research code validates features/splits/provenance; Random Forest artifact is serialized. Endpoint-native training is not present. Status: preserved CERT artifact; original raw training data unavailable.



12. ML inference

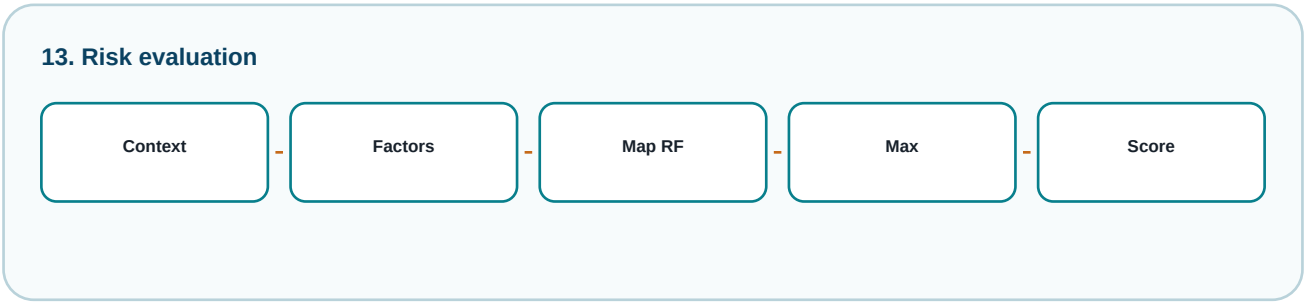
Complete named aggregate passes Node/Python invariants; Python loads joblib model and SHAP; fingerprints/threshold/provenance return to Node; prediction can enter risk history. Status: automated-test verified; not live

endpoint inference.



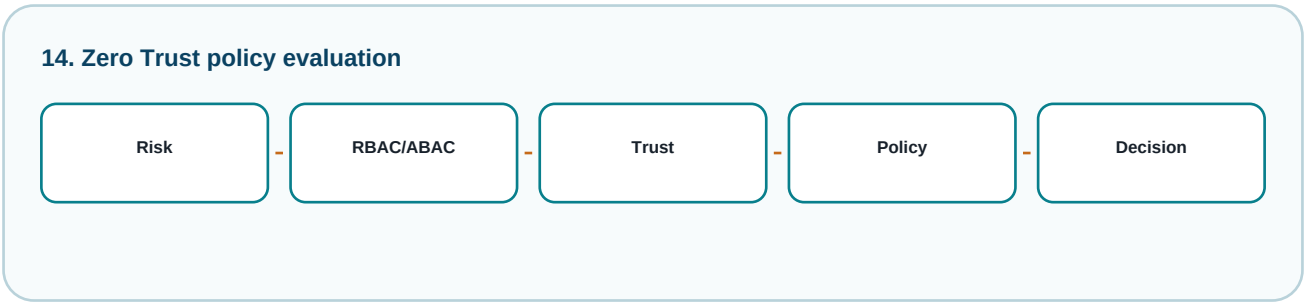
13. Risk evaluation

Context factors are built, strongest factor per category is retained, contextual evidence is capped, optional RF evidence is threshold-mapped, and max evidence becomes bounded risk. Status: automated-test verified.



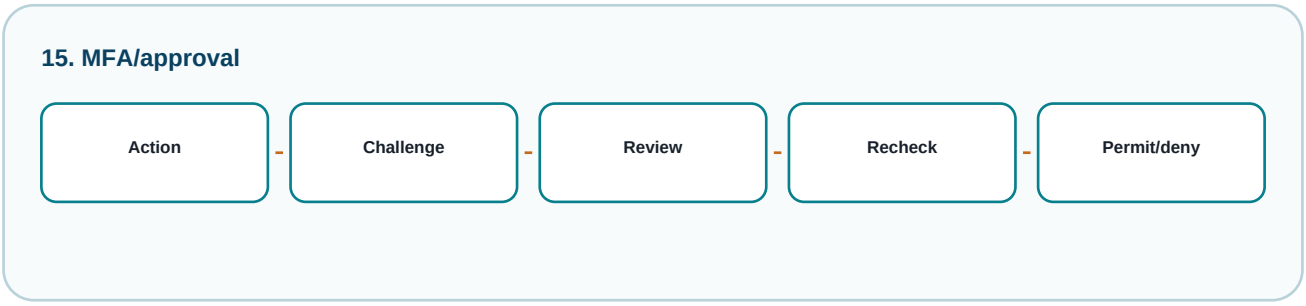
14. Zero Trust policy evaluation

Risk score plus role, resource sensitivity, trust, account status, incident holds and approvals are evaluated; policy returns ALLOW/MONITOR/MFA/approval/restriction/freeze. Status: automated-test verified.



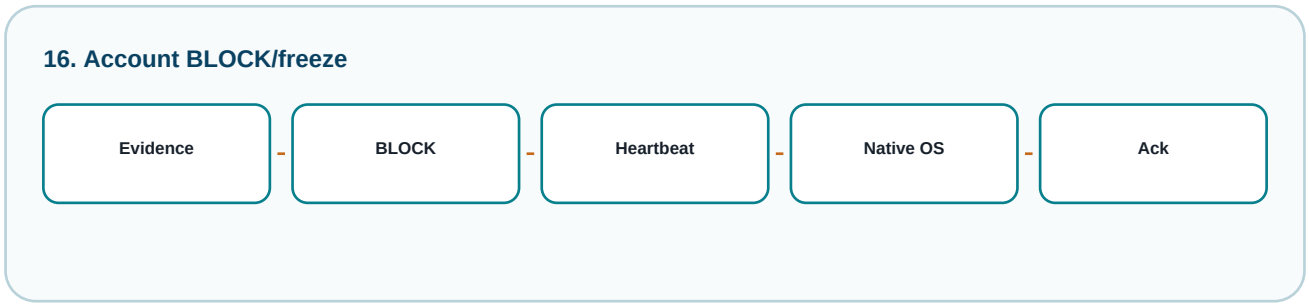
15. MFA/approval

MFA action creates/validates fresh OTP or approval route records reviewer decision; protected action is re-evaluated and cannot bypass role/hold constraints. Status: automated-test verified.



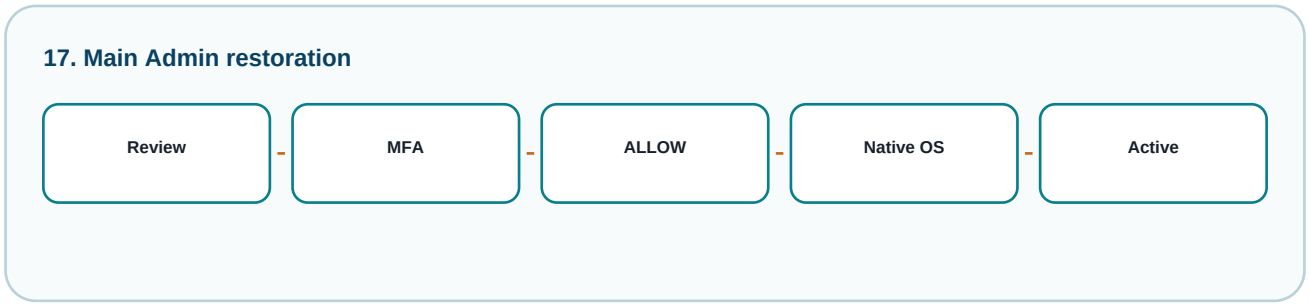
16. Account BLOCK/freeze

Eligible fresh protected access creates expiring signed BLOCK; heartbeat delivers; agent verifies binding/replay/expiry; native helper reports BLOCKED or failure. Status: workflow automated-tested; physical OS unverified.



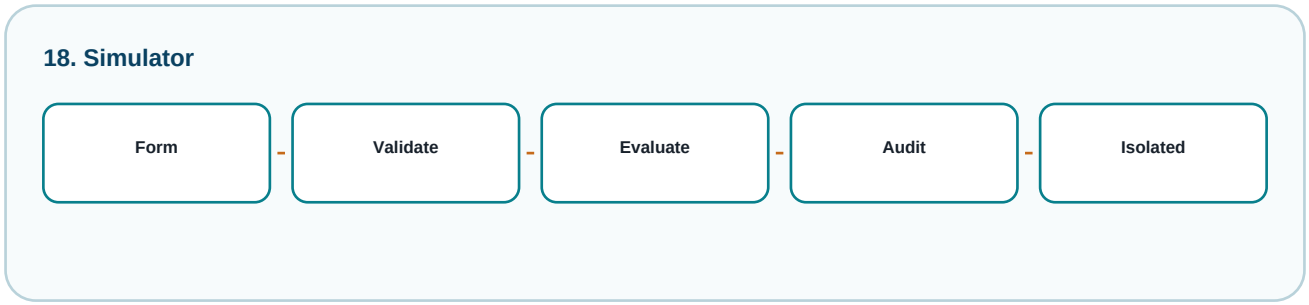
17. Main Admin restoration

Reviewed hold plus fresh Main Admin MFA issues ALLOW; agent verifies and native helper reports ALLOWED; backend restores only after required acknowledgments. Status: workflow automated-tested; physical OS unverified.



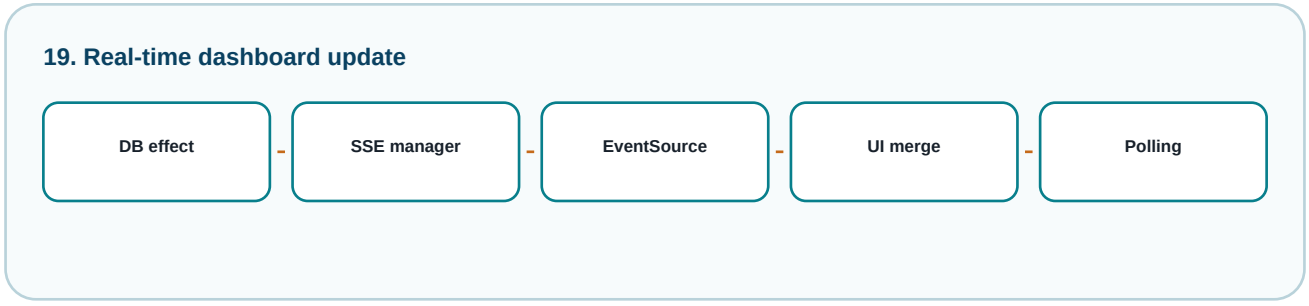
18. Simulator

Admin submits bounded scenario/custom inputs; dedicated route returns simulation-only evaluation and audit history; tests assert no employee telemetry/risk/incident/command/account mutation. Status: simulated and automated-test verified.



19. Real-time dashboard update

Database side effect broadcasts through in-process SseManager after authorization rechecks; EventSource updates UI; persisted notification polling provides fallback. Status: source/automated/browser fixture verified; production delivery unverified.



Pipeline evidence matrix

Pipeline	Primary source evidence	Current status
Login/enrollment	server/routes/auth.js; server/routes/endpoint.js; agent/src/enrollment.js	Automated-test verified
Telemetry/USB/application	agent/src/service.js; agent/src/collectors/*; server/services/endpoint-service.js	Contract tested; physical gaps
ML training/inference	ml/src/*; ml/models/*; server/services/ml-service.js	CERT preserved; endpoint-native unavailable
Risk/policy	cyber/risk-engine/risk-engine.js; server/services/policy-service.js	Automated-test verified
Containment/restoration	server/services/security-service.js; agent/src/account-control.js	Automated workflow; OS unverified
Notifications/SSE	server/services/sse.js; client/src/App.jsx	Persisted/SSE/polling tested